

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

DESARROLLO Y OPTIMIZACIÓN DE UN
SISTEMA MODULAR DE CÓDIGO ABIERTO
PARA LA PRODUCCIÓN REMOTA DE
VÍDEO EN TIEMPO REAL

MARTÍN HERRANZ SÁNCHEZ

Junio 2026

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Título: Desarrollo y optimización de un sistema modular de código abierto para la producción remota de vídeo en tiempo real

Autor: D. Martín Herranz Sánchez

Tutor: D. Álvaro Llorente Gómez

Ponente: D. David Jiménez Bermejo

Departamento: Electrónica física, Ingeniería Eléctrica y Física Aplicada

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

Madrid, a de de 2026

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE
TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

**Desarrollo y optimización de un
sistema modular de código abierto
para la producción remota de vídeo
en tiempo real**

Martín Herranz Sánchez

Junio 2026

Resumen

La producción audiovisual en directo ha evolucionado desde grandes estudios con equipamiento dedicado de alto coste hacia soluciones basadas en software que permiten desplegar un sistema profesional de realización con hardware convencional. Voctomix es un mezclador de vídeo de código abierto desarrollado sobre GStreamer y Python que posibilita esta democratización de la producción en directo.

Este Trabajo Fin de Grado presenta el diseño e implementación de Voctomix 2.0, una versión extendida del mezclador original con nuevas capacidades orientadas a la producción remota. Las contribuciones principales del proyecto son: la integración de un sistema de gestión de composites avanzados (PIP, side-by-side, fullscreen), un módulo de transiciones basadas en mezcla de señales, soporte para overlays gráficos con animaciones de entrada y salida, gestión dinámica del audio mediante un servicio dedicado, y un sistema de telemetría basado en RabbitMQ que registra en tiempo real todos los eventos de sesión.

Para facilitar el despliegue reproducible del sistema completo, se ha desarrollado una imagen Docker junto con un fichero Docker Compose que orquesta voctocore, las fuentes de prueba FFmpeg, el broker RabbitMQ y el servicio de telemetría en una red privada aislada. Este despliegue permite reproducir el entorno completo en cualquier máquina con un único comando.

El sistema ha sido validado en dos escenarios: un despliegue en dos equipos físicos en red local y un despliegue completo en contenedores Docker. Los resultados confirman una latencia de conmutación inferior a un fotograma (<40 ms) y el registro correcto de todos los eventos de sesión en RabbitMQ.

Summary

Live audiovisual production has evolved from large studios relying on dedicated high-cost hardware towards software-based solutions that allow deploying a professional production system on commodity hardware. Voctomix is an open-source video mixer built on GStreamer and Python that enables this democratisation of live production.

This Bachelor's Thesis presents the design and implementation of Voctomix 2.0, an extended version of the original mixer featuring new capabilities aimed at remote production. The main contributions of the project are: integration of an advanced composite management system (PIP, side-by-side, fullscreen), a signal-mixing-based transitions module, support for graphical overlays with blend-in and blend-out animations, dynamic audio management through a dedicated service, and a RabbitMQ-based telemetry system that records all session events in real time.

To facilitate reproducible deployment of the full system, a Docker image and a Docker Compose file have been developed, orchestrating voctocore, FFmpeg test sources, the RabbitMQ broker, and the telemetry service in an isolated private network. This deployment allows reproducing the complete environment on any machine with a single command.

The system has been validated in two scenarios: a two-physical-machine deployment over a local network and a full Docker container deployment. Results confirm a switching latency below one video frame (<40 ms) and correct logging of all session events in RabbitMQ.

Palabras clave

producción audiovisual en directo, mezclador de vídeo, GStreamer, Docker, RabbitMQ, telemetría, software libre, *streaming*, AMQP.

Keywords

live audiovisual production, video mixer, GStreamer, Docker, RabbitMQ, telemetry, free software, streaming, AMQP.

Agradecimientos

En primer lugar, quiero agradecer a mi tutor [Nombre del tutor] su orientación, disponibilidad y los consejos que han guiado el desarrollo de este trabajo desde el primer día. Su apoyo ha sido fundamental para mantener el rumbo del proyecto en los momentos de mayor dificultad.

A mis compañeros de la ETSIT, con quienes he compartido estos años de carrera: gracias por los apuntes prestados, los proyectos en común y, sobre todo, por las horas de estudio que han hecho más llevadero el camino.

A mi familia, por su paciencia infinita durante los periodos de mayor carga de trabajo y por creer siempre en mí incluso cuando yo dudaba. Sin vuestro apoyo, este proyecto no habría sido posible.

Por último, a la comunidad de software libre y, en especial, a los desarrolladores y colaboradores de Voctomix y GStreamer, cuyo trabajo desinteresado es la base sobre la que se sustenta este TFG.

Índice

Resumen	II
Summary	III
Palabras clave	III
Keywords	III
Agradecimientos	IV
Lista de acrónimos	XI
1. Introducción y objetivos	1
1.1. Introducción	1
1.2. Objetivos	2
1.3. Estructura de la memoria	2
2. Estado del arte	4
2.1. Producción audiovisual	4
2.1.1. Producción audiovisual tradicional	4
2.1.2. Producción audiovisual remota	5
2.2. Protocolos de transporte de red	8
2.2.1. UDP (<i>User Datagram Protocol</i>)	8
2.2.2. TCP (<i>Transmission Control Protocol</i>)	9
2.2.3. HTTP / HTTPS (<i>Hypertext Transfer Protocol</i>)	9
2.3. Contribución Audiovisual	10
2.3.1. Protocolos de contribución	10
2.3.2. Formatos de contribución	11
2.3.3. Códecs de contribución	11
2.4. Distribución Audiovisual	12
2.4.1. Protocolos de distribución	13
2.4.2. Formatos de distribución	13
2.4.3. Códecs de distribución	14
2.5. Casos de uso reales de producción remota	15
2.5.1. Chaos Communication Congress (C3VOC)	15
2.5.2. Juegos Olímpicos de Tokio 2020 (OBS)	15
2.5.3. UEFA Champions League (Telefónica Servicios Audiovisuales)	16
2.5.4. Proyecto CyberNEMO (Universidad Politécnica de Madrid)	16
3. Diseño e implementación del sistema de producción remota de vídeo	17
3.1. Arquitectura del sistema	19
3.1.1. voctocore: núcleo multimedia	19

3.1.2.	votogui: interfaz gráfica de control	19
3.2.	Fuentes de señal de cámaras	20
3.2.1.	Conexión de las fuentes	20
3.2.2.	Previsualización en la interfaz gráfica	21
3.2.3.	Requisitos de formato de la señal de entrada	22
3.3.	Fuentes de señal auxiliares	23
3.3.1.	Fuente break e intro	23
3.3.2.	Entrada de audio	24
3.3.3.	Stream Blanker: estados de emisión	24
3.4.	Composites	25
3.4.1.	Full Screen (fs)	26
3.4.2.	Picture in Picture (pip)	26
3.4.3.	Side by Side (sbs)	26
3.4.4.	Lecture (lec)	26
3.4.5.	Mirror	26
3.5.	Transiciones	27
3.5.1.	Cut (corte instantáneo)	27
3.5.2.	Trans (transición suave)	27
3.5.3.	Retake	27
3.6.	Overlays, logos y texto dinámico	28
3.6.1.	Logos	28
3.6.2.	Overlays PNG y texto dinámico	28
3.6.3.	Rotulación dinámica de ponente	29
3.6.4.	Sistema de doble rotulación	29
3.6.5.	Botón UPDATE: recarga dinámica de recursos	30
3.6.6.	Auto-off de overlays	30
3.7.	Señal de programa final	30
3.8.	Personalización de la interfaz gráfica	31
3.8.1.	Tema oscuro y hoja de estilos	31
3.8.2.	Iconografía SVG	32
3.8.3.	Reloj de estudio	32
3.9.	Sistema de telemetría	32
3.9.1.	Arquitectura del servicio de telemetría	32
3.9.2.	API REST y mensajería AMQP	34
3.9.3.	RabbitMQ como broker de mensajería	35
3.9.4.	Evolución del protocolo de telemetría: de UDP a HTTP	36
4.	Despliegue del sistema	37
4.1.	Scripts de arranque	37
4.1.1.	Escenario monopuesto (<code>start_studio_single_pc.sh</code>)	37
4.1.2.	Escenario dos PCs (<code>2pc_escenario2/</code>)	37
4.1.3.	Escenario Docker (<code>launch_docker_studio.sh</code>)	37
4.1.4.	Escenario Kubernetes (<code>launch_k8s.sh</code>)	38
4.2.	Despliegue con Docker Compose	38
4.2.1.	Dockerfile	38
4.2.2.	Docker Compose y espacio de red compartido	39
4.2.3.	GUI nativa y volúmenes	39
4.2.4.	Resiliencia al arranque	39

4.3.	Despliegue en Kubernetes	40
4.3.1.	Estructura de manifiestos	40
4.3.2.	Orden de arranque y sondas de preparación	41
4.3.3.	Script de lanzamiento	41
5.	Pruebas de validación del sistema de producción remota de vídeo	42
5.1.	Entorno de pruebas	42
5.2.	Validación del sistema de telemetría	43
5.3.	Rendimiento en función del número de cámaras	44
5.4.	Análisis de rendimiento por escenario de despliegue	45
5.4.1.	Consumo de CPU y memoria	45
5.4.2.	Ancho de banda interno	46
5.4.3.	Latencia de conmutación	46
5.5.	Matriz de funcionalidades por escenario	47
6.	Conclusiones y líneas futuras	48
6.1.	Conclusiones	48
6.2.	Líneas de trabajo futuro	50
	Bibliografía	52
	Anexo A: Aspectos éticos, económicos, sociales y ambientales	55
	Anexo B: Presupuesto económico	59

Índice de figuras

2.1. Arquitectura del modelo REMI.	5
2.2. Cadena de producción audiovisual en directo: señales de contribución, mezclador de vídeo y salida de distribución.	8
3.1. Arquitectura global del sistema Voctomix 2.0.	18
3.2. Interfaz gráfica completa de Voctomix 2.0 (voctogui).	20
3.3. Panel de cámaras en voctogui.	22
3.4. Estados NOSTREAM y PAUSE del stream blanker: vista del realizador (arriba) y señal recibida por el espectador (abajo).	25
3.5. Disposición visual de las fuentes A y B en los cuatro composites principales de Voctomix 2.0: pantalla completa (fs), imagen en imagen (pip), lado a lado (sbs) y conferencia (lec).	25
3.6. Selector de modos de composite en la interfaz de Voctomix 2.0.	27
3.7. Botones de transición en la interfaz de Voctomix 2.0: CUT, TRANS y RETAKE.	28
3.8. Panel de overlays y rotulación dinámica en la interfaz de Voctomix 2.0. . .	30
3.9. Interfaz gráfica voctogui personalizada	31
3.10. Colas RabbitMQ activas durante la integración con CyberNEMO.	35
4.1. Arquitectura de pods en el despliegue Kubernetes.	41
5.1. Cola <code>voctomix_events</code> en la consola RabbitMQ tras la sesión de prueba. . .	43
5.2. Salida en tiempo real del terminal: evento STATE y CHANGE.	44
5.3. Consumo de CPU del sistema Docker en función del número de cámaras activas.	44
5.4. Consumo de RAM de voctocore en función del número de cámaras activas. .	45
5.5. Consumo energético estimado en función del número de cámaras activas. .	45
5.6. Consumo medio de CPU y RAM de voctocore por escenario de despliegue, con cuatro fuentes activas.	46
5.7. Distribución de latencias de conmutación en el escenario de dos PCs (WiFi 5 GHz), $n = 20$ mediciones. El 100 % de las mediciones fue inferior a un fotograma a 25 fps (40 ms).	47

Índice de tablas

2.1. Beneficios y desafíos de la producción audiovisual remota (REMI).	6
2.2. Comparativa entre producción audiovisual tradicional y producción remota (REMI).	7
2.3. Comparativa de los protocolos de transporte en producción audiovisual. . .	9

2.4.	Comparativa de protocolos de contribución audiovisual.	11
2.5.	Comparativa de códecs de contribución audiovisual.	12
2.6.	Comparativa de protocolos de distribución audiovisual en directo.	13
2.7.	Comparativa de códecs de vídeo para distribución en directo.	15
2.8.	Casos de uso representativos de producción remota audiovisual.	16
3.1.	Puertos del sistema Voctomix 2.0.	19
4.1.	Scripts de arranque por escenario de despliegue.	38
4.2.	Servicios definidos en <code>docker-compose.yml</code>	39
5.1.	Especificaciones del equipo empleado en las pruebas.	42
5.2.	Ancho de banda interno estimado según número de fuentes activas.	46
5.3.	Validación de funcionalidades del sistema por escenario de despliegue.	47
6.1.	Consecución de los objetivos específicos del TFG.	49
6.2.	Presupuesto económico	59

Lista de acrónimos

ABR	Adaptive Bitrate (tasa de bits adaptativa)
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AVC	Advanced Video Coding (véase H.264/MPEG-4 AVC)
CDN	Content Delivery Network (red de distribución de contenido)
CPU	Central Processing Unit
DASH	Dynamic Adaptive Streaming over HTTP
FLOSS	Free/Libre and Open Source Software
FPS	Frames Per Second (fotogramas por segundo)
GPU	Graphics Processing Unit
GUI	Graphical User Interface (interfaz gráfica de usuario)
HEVC	High Efficiency Video Coding (véase H.265)
HLS	HTTP Live Streaming
HTTP	HyperText Transfer Protocol
ITU-T	International Telecommunication Union – Telecommunication Standardization Sector
JSON	JavaScript Object Notation
MPEG	Moving Picture Experts Group
NDI	Network Device Interface
OBS	Open Broadcaster Software
PIP	Picture-in-Picture
REST	Representational State Transfer
RTMP	Real-Time Messaging Protocol
RTSP	Real-Time Streaming Protocol
SDI	Serial Digital Interface
SMPTE	Society of Motion Picture and Television Engineers
SRT	Secure Reliable Transport
TCP	Transmission Control Protocol
TFG	Trabajo Fin de Grado
UDP	User Datagram Protocol
VBR	Variable Bitrate (tasa de bits variable)
VP9	Video codec desarrollado por Google (estándar abierto)
YAML	YAML Ain't Markup Language

1. Introducción y objetivos

1.1. Introducción

La producción audiovisual en directo ha experimentado en los últimos años una profunda transformación, impulsada por la democratización del hardware de bajo coste y el crecimiento exponencial del *streaming* (transmisión en flujo) en Internet. Eventos técnicos de referencia como el Chaos Communication Congress (CCC), organizado por el Chaos Computer Club en Alemania, han puesto de manifiesto la necesidad de disponer de herramientas de realización profesional que sean al mismo tiempo económicas, fiables y auditables.

En este contexto surge Voctomix, desarrollado por el equipo C3VOC (*Chaos Communication Congress Video Operations Crew*). Frente a soluciones previas como *dvs*switch, limitada en resolución y escalabilidad, o herramientas comerciales como vMix o OBS Studio, diseñadas para un único operador en un único equipo, Voctomix adopta una arquitectura cliente-servidor desacoplada que separa el motor de mezcla multimedia de la interfaz de control, permitiendo distribuir la carga de procesamiento en distintos nodos de red.

El presente Trabajo de Fin de Grado toma como punto de partida la versión original de Voctomix y propone una serie de extensiones orientadas a su uso en producciones en directo de mediana escala: rotulación dinámica sobre el vídeo, gestión avanzada del blanking de stream, composites avanzados de vídeo, telemetría en tiempo real mediante un broker de mensajería, y la contenerización completa del sistema mediante Docker y Docker Compose. El resultado es Voctomix 2.0, una plataforma más versátil, desplegable y adaptada a flujos de trabajo profesionales.

Un elemento diferenciador de este proyecto es su orientación hacia un **caso de uso real**: el sistema desarrollado se integra en el proyecto europeo de investigación **CyberNEMO**, liderado por la Universidad Politécnica de Madrid (UPM), cuyo objetivo es validar soluciones de ciberseguridad aplicadas a entornos de producción y distribución multimedia. En este marco, Voctomix 2.0 actúa como el motor de producción audiovisual del proyecto, mientras que el sistema de telemetría con RabbitMQ desarrollado en este trabajo proporciona el canal de monitorización en tiempo real del *pipeline* (cadena de procesamiento) de producción. Voctomix 2.0 responde directamente a las necesidades de retransmisión de seminarios y eventos técnicos con calidad profesional sin depender de infraestructura hardware costosa, proporcionando un entorno reproducible que puede ponerse en funcionamiento con un único comando (`docker compose up`) en cualquier equipo Linux estándar.

1.2. Objetivos

El objetivo principal de este trabajo es diseñar e implementar una arquitectura de código abierto, modular y reproducible para la producción remota de vídeo en tiempo real. Partiendo de Voctomix como base, se busca construir un sistema completo que permita realizar producciones audiovisuales de calidad profesional sin depender de hardware propietario de alto coste, y que pueda desplegarse fácilmente mediante contenedores.

Para lograrlo, se plantean los siguientes objetivos específicos:

1. Ampliar las funcionalidades del mezclador con características propias de la producción profesional: rotulación dinámica con gráficos y textos en tiempo real, un gestor de estados de emisión y sincronización automática de audio con el vídeo.
2. Implementar un servicio de telemetría que exporte el estado del sistema en formato JSON y lo publique en un broker de mensajería, de forma que se pueda integrar con herramientas externas sin tocar el núcleo de la aplicación.
3. Contenerizar el sistema completo con Docker, de modo que cualquiera pueda desplegarlo con un único comando y sin preocuparse por dependencias ni configuraciones del entorno.
4. Extender el despliegue a Kubernetes, validando que el sistema funciona también en infraestructuras más complejas y multi-nodo.
5. Validar el funcionamiento del sistema en varios escenarios reales: desde un único ordenador hasta un clúster de Kubernetes, comprobando que todo funciona correctamente en cada uno de ellos.
6. Demostrar que el sistema es útil en producción real, a través de su uso en el proyecto europeo CyberNEMO en la Universidad Politécnica de Madrid.

1.3. Estructura de la memoria

La presente memoria se organiza en seis capítulos y dos anexos:

El **Capítulo 2** presenta el estado del arte en producción audiovisual: la evolución desde la producción tradicional hasta el modelo remoto basado en software, los protocolos de red y los estándares de contribución y distribución de vídeo, y una comparativa de los mezcladores disponibles. Se incluyen también casos de uso reales de producción remota a escala industrial.

El **Capítulo 3** describe el diseño e implementación del sistema Voctomix 2.0: la arquitectura cliente-servidor (voctocore y voctogui), las fuentes de señal de cámaras y auxiliares, los modos de composición, las transiciones, el sistema de *overlays* y rotulación dinámica, y el servicio de telemetría mediante RabbitMQ.

El **Capítulo 4** detalla el despliegue del sistema en sus distintos escenarios: los scripts de arranque para entorno nativo, el despliegue contenerizado con Docker Compose y el despliegue orquestado con Kubernetes.

El **Capítulo 5** recoge las pruebas de validación del sistema en los escenarios de despliegue,

con mediciones de latencia, registros de telemetría y evidencias del correcto funcionamiento.

El **Capítulo 6** presenta las conclusiones del trabajo y las líneas de trabajo futuro identificadas.

Los **Anexos** incluyen los aspectos éticos, económicos, sociales y ambientales del proyecto (Anexo A) y el presupuesto económico detallado (Anexo B).

2. Estado del arte

En este capítulo se presentan las bases tecnológicas sobre las que se sustenta Voctomix 2.0. El capítulo se estructura en cinco secciones: el modelo de producción audiovisual y las herramientas de realización disponibles (Sección 2.1), los protocolos de transporte de red que subyacen a todos los flujos multimedia (Sección 2.2), los estándares e interfaces de contribución de señal (Sección 2.3), los protocolos y códecs empleados en la distribución del contenido al espectador final (Sección 2.4), y los casos de uso reales de producción remota que contextualizan la adopción industrial de estas tecnologías (Sección 2.5). El objetivo es que el lector comprenda el contexto técnico en el que se enmarca el trabajo antes de abordar el diseño e implementación del sistema en el Capítulo 3.

2.1. Producción audiovisual

La producción audiovisual en directo engloba el conjunto de procesos técnicos y organizativos que permiten capturar, procesar y distribuir contenido de vídeo y audio en tiempo real. Históricamente ligada a infraestructuras físicas costosas y al desplazamiento de equipos al lugar del evento, esta disciplina ha cambiado profundamente en la última década gracias a la integración de las redes IP en los flujos de producción profesional [1].

2.1.1. Producción audiovisual tradicional

El modelo de producción tradicional se basa en el despliegue físico de toda la infraestructura técnica y del personal en el emplazamiento del evento. El componente central de este modelo es el **mezclador hardware**, denominado *switcher* (conmutador de vídeo), un dispositivo dedicado que selecciona en cada instante qué fuente de vídeo se emite a la salida de programa y gestiona las transiciones y los efectos. Fabricantes como Blackmagic Design (ATEM), Ross Video o Grass Valley (Kayenne) dominan el mercado de estos equipos, que ofrecen latencias sub-imagen, múltiples buses de programa y previsualización simultáneos, y redundancia hardware certificada para operación continua [2].

En el modelo tradicional, todas las fuentes, cámaras, reproductores de vídeo y ordenadores de presentación, se interconectan mediante cableado **SDI** (*Serial Digital Interface*), el estándar histórico de la industria *broadcast* (radiodifusión) para el transporte de vídeo sin comprimir [3]. La señal resultante recorre el mezclador, los sistemas de rotulación y grabación, y finalmente el encoder de distribución, todo ello dentro del mismo emplazamiento físico.

Este modelo ha demostrado su solidez durante décadas en las grandes producciones de televisión: retransmisiones deportivas, eventos parlamentarios y galas de premios emplean

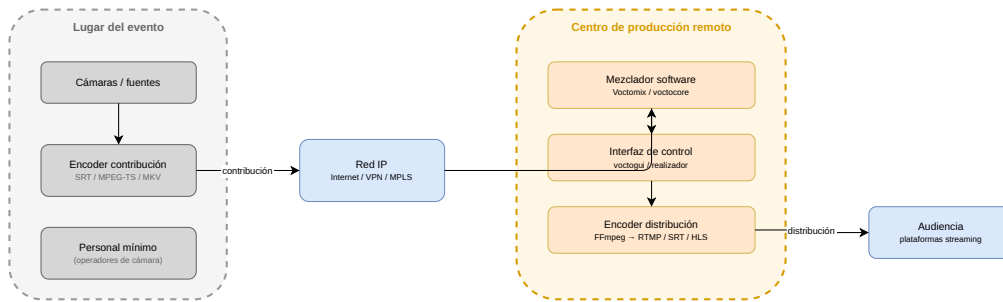


Figura 2.1: Arquitectura del modelo REMI.

esta arquitectura con decenas de fuentes de cámara gestionadas en tiempo real. Sin embargo, su principal limitación es económica y logística: el coste de adquirir y transportar el equipamiento, más el personal técnico necesario para su instalación y operación, lo pone fuera del alcance de organizaciones de mediana escala.

Un caso ilustrativo es la retransmisión de un partido de La Liga de fútbol profesional en España. Este tipo de producción moviliza habitualmente entre 20 y 30 cámaras SDI, varias unidades móviles de producción equipadas con mezcladores hardware de gama alta y un equipo técnico de entre 50 y 100 personas desplazadas al estadio [2]. El coste de una unidad móvil de estas características puede superar los 2 millones de euros, cifra a la que se suma el gasto recurrente en desplazamientos, montaje y operación. Este modelo queda así reservado a grandes cadenas de televisión y organizaciones con infraestructura broadcast consolidada, resultando inaccesible para entidades académicas, asociaciones o eventos de mediana escala.

2.1.2. Producción audiovisual remota

La producción remota, conocida en la industria por el acrónimo **REMI** (*Remote Integration Model*), surge para resolver estas limitaciones permitiendo que la realización se lleve a cabo desde un centro de producción centralizado, mientras en el lugar del evento solo se mantiene el equipo técnico mínimo necesario para la captura de señal [1, 4].

Arquitectura técnica del modelo REMI

La figura 2.1 ilustra la separación funcional entre el lugar del evento y el centro de producción remoto. En el emplazamiento solo permanecen las cámaras, los micrófonos y un sistema de contribución IP que codifica y transmite la señal hacia el centro remoto. El procesamiento completo, mezcla de fuentes, inserción de gráficos, control de audio, gestión de overlays y distribución al espectador, se realiza de forma centralizada.

Esta separación permite que el realizador opere el sistema desde cualquier ubicación con acceso a la red, y que el centro de producción pueda atender simultáneamente eventos en distintas localizaciones reutilizando la misma infraestructura de mezclado [4].

El modelo REMI se ha consolidado en los últimos años como referencia en eventos de gran escala. Los casos representativos, desde los Juegos Olímpicos de Tokio 2020 hasta el Chaos Communication Congress, se analizan en la Sección 2.5.

Ventajas del modelo remoto

Las ventajas del modelo REMI frente a la producción tradicional son significativas [1, 4]:

- **Reducción de costes:** se minimiza el desplazamiento de personal y equipos. Solo se requiere *in situ* el equipo de captura; la realización, mezcla y edición se realizan de forma centralizada.
- **Sostenibilidad:** la reducción de desplazamientos contribuye a disminuir la huella de carbono de las producciones, un factor cada vez más valorado por organizaciones y broadcasters.
- **Escalabilidad:** el sistema puede adaptarse a producciones de diferente escala reutilizando la misma infraestructura centralizada.
- **Flexibilidad geográfica:** es posible retransmitir eventos desde cualquier parte del mundo con conectividad IP, sin necesidad de desplazar el equipo de producción.

La adopción del modelo REMI no está exenta de retos. La Tabla 2.1 resume los principales beneficios y desafíos identificados en entornos de producción reales [1, 4].

Tabla 2.1: Beneficios y desafíos de la producción audiovisual remota (REMI).

Dimensión	Beneficio	Desafío / Dificultad
Coste	Reducción de desplazamientos y equipos	Inversión inicial en red y software
Latencia	Viable con SRT (<1 s en WAN)	Coordinación realizador-cámaras difícil
Escalabilidad	Un centro atiende múltiples eventos	Riesgo de sobrecarga del servidor
Red	Producción desde cualquier ubicación IP	Corte de red detiene la producción
Calidad	Hardware ilimitado en el centro remoto	Artefactos con ancho de banda insuficiente
Sincronía A/V	MKV + SRT garantizan sincronía	<i>jitter</i> (fluctuación) puede desincronizar fuentes
Seguridad	Cifrado AES-256 extremo a extremo	Mayor superficie de ataque expuesta
Complejidad	Herramientas maduras (Voctomix, FFmpeg)	Diagnóstico de pipelines distribuidos
Sostenibilidad	Menor huella de carbono	Consumo energético del centro permanente

Herramientas de producción remota por software

La disponibilidad de herramientas de mezclado por software ha democratizado el acceso al modelo remoto incluso para organizaciones sin presupuesto broadcast:

- **OBS Studio** [5]: solución de código abierto ampliamente utilizada para *streaming* y grabación. Su diseño todo en uno concentra captura, composición y emisión en una única aplicación para un único operador. Soporta fuentes de entrada variadas, como cámaras USB, captura de pantalla o fuentes IP, con soporte para transiciones y filtros. Es la opción más utilizada por *streamers* independientes, aunque su diseño centralizado limita su uso en entornos distribuidos.
- **vMix** [6]: solución comercial profesional para Windows con soporte NDI, *multi-view* (vista múltiple) y producción 4K. Permite gestionar múltiples fuentes, aplicar *chroma key* (croma) y transiciones, y emitir a varias plataformas simultáneamente. Su coste de licencia, que puede superar los 1.000 €, y su restricción a Windows la excluyen de entornos de código abierto sobre Linux.
- **Voctomix** [7]: herramienta de código abierto con arquitectura cliente-servidor que separa el motor de mezcla (**voctocore**) de la interfaz de control (**voctogui**), permitiendo que el procesamiento se realice en un servidor mientras el operador trabaja desde otro equipo conectado por red. Construido sobre GStreamer [8], un *framework* (entorno de software) multimedia de código abierto, proporciona composición de vídeo con múltiples fuentes, transiciones animadas, *overlays* y un protocolo de control TCP extensible. Es la herramienta objeto del presente trabajo (véase el Capítulo 3).

Tabla 2.2: Comparativa entre producción audiovisual tradicional y producción remota (REMI).

Parámetro	Tradicional	REMI / Software
Infraestructura en lugar del evento	Completa	Mínima (cámaras + encoder)
Coste de equipamiento	Muy alto	Bajo (hardware convencional)
Personal <i>in situ</i>	Numeroso	Reducido
Latencia extremo a extremo	Sub-imagen	1 fotograma – varios segundos
Escalabilidad	Limitada	Alta
Sostenibilidad (huella de carbono)	Baja	Alta
Ejemplo	Blackmagic ATEM	OBS, vMix, Voctomix

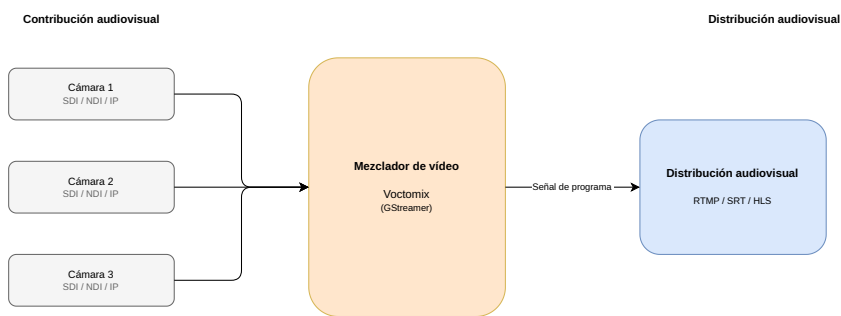


Figura 2.2: Cadena de producción audiovisual en directo: señales de contribución, mezclador de vídeo y salida de distribución.

2.2. Protocolos de transporte de red

Los protocolos de aplicación que gobiernan la contribución y la distribución audiovisual se construyen sobre un conjunto reducido de protocolos de transporte de red. Comprender su comportamiento es fundamental para interpretar las decisiones de diseño que subyacen a cada solución de producción, ya que sus propiedades determinan directamente la latencia, la fiabilidad y la escalabilidad del sistema.

2.2.1. UDP (*User Datagram Protocol*)

UDP es un protocolo de transporte sin conexión definido en la RFC 768 [9]. Envía datagramas de forma independiente sin establecer sesión previa, sin confirmación de entrega, sin reordenación de paquetes y sin control de congestión. Estas características le confieren la mínima latencia posible dentro de la pila de red, al eliminar todo el *overhead* de fiabilidad.

En producción audiovisual, UDP es el protocolo de elección cuando la latencia prima sobre la integridad de los datos. Un fotograma perdido es preferible a un fotograma retrasado en aplicaciones de tiempo real. Por este motivo, los siguientes protocolos de nivel superior operan sobre UDP:

- **RTP** (*Real-time Transport Protocol*): encapsula los flujos multimedia con marcas de tiempo y números de secuencia que permiten al receptor detectar y compensar pérdidas sin retransmisión.
- **WebRTC**: emplea UDP con DTLS/SRTP para comunicaciones en tiempo real con latencias de 100–500 ms, asumiendo pérdidas ocasionales de paquetes.
- **SRT**: utiliza UDP como transporte pero añade una capa ARQ (*Automatic Repeat Request*) propia para recuperar paquetes perdidos sin incrementar la latencia en condiciones normales de red.

2.2.2. TCP (*Transmission Control Protocol*)

TCP es un protocolo de transporte orientado a conexión definido en la RFC 793 [10]. Garantiza la entrega ordenada de todos los bytes mediante confirmaciones de recepción, retransmisión automática de segmentos perdidos y control de flujo y congestión. Estas garantías introducen latencia variable en redes con pérdidas, ya que el receptor debe esperar la retransmisión antes de procesar datos posteriores.

En producción audiovisual, TCP se emplea cuando la integridad de los datos no es negociable y la latencia adicional es aceptable:

- **RTMP**: protocolo de ingesta basado en TCP que prioriza la entrega completa de cada fotograma sobre plataformas como YouTube Live o Twitch.
- **Voctomix interno**: el núcleo voctocore acepta las fuentes de cámara mediante conexiones TCP que transportan flujos MKV con vídeo RAW I420 y audio PCM S16LE. TCP garantiza la sincronización audio-vídeo sin pérdidas dentro de la red local.
- **Protocolo de control**: el servidor de comandos de voctocore escucha en el puerto 9999 mediante TCP, asegurando que cada comando enviado por la interfaz gráfica llega íntegramente al núcleo y que la respuesta de estado retorna sin corrupción.

2.2.3. HTTP / HTTPS (*Hypertext Transfer Protocol*)

HTTP es un protocolo de capa de aplicación definido en la RFC 9110 [11] que opera sobre TCP (HTTP/1.1 y HTTP/2) o sobre QUIC/UDP (HTTP/3). Su modelo de petición-respuesta y su infraestructura de caché y distribución global mediante CDN (*Content Delivery Networks*) lo convierten en el protocolo de referencia para la distribución de contenidos a audiencias masivas.

En producción audiovisual, HTTP actúa como capa de transporte para los principales protocolos de distribución adaptativa:

- **HLS y MPEG-DASH**: dividen el vídeo en segmentos de fichero servidos como respuestas HTTP estándar. Cualquier infraestructura de servidores web o CDN puede distribuir estos segmentos sin configuración específica de media.
- **API REST de telemetría**: el servicio de telemetría de Voctomix 2.0 expone el estado de la sesión mediante HTTP en el puerto 8080, y el módulo de exportación de la GUI envía actualizaciones de estado mediante peticiones HTTP POST.

Tabla 2.3: Comparativa de los protocolos de transporte en producción audiovisual.

Protocolo	Orientado a conexión	Fiabilidad	Latencia	Uso en Voctomix
UDP	No	Sin garantías	Mínima	RTP, SRT, WebRTC
TCP	Sí	Garantizada	Variable	MKV/fuentes, RTMP, control
HTTP	Sí*	Garantizada	Alta	HLS, DASH, telemetría REST

*HTTP/1.1 y HTTP/2 operan sobre TCP; HTTP/3 opera sobre QUIC (UDP).

2.3. Contribución Audiovisual

La contribución audiovisual designa el transporte de la señal de vídeo y audio desde las fuentes de captura, cámaras, ordenadores de presentación y reproductores, hasta el mezclador o núcleo del sistema de producción. Es el primer eslabón de la cadena y su elección determina la calidad máxima disponible, la latencia de extremo a extremo y el coste de la infraestructura. En esta sección se describen los protocolos, formatos contenedor y códecs empleados en esta etapa.

2.3.1. Protocolos de contribución

Los protocolos de contribución definen el mecanismo de transporte de la señal desde el origen hasta el mezclador. La elección depende del entorno: producción en estudio, red local o contribución remota sobre Internet.

- **SDI (*Serial Digital Interface*)**: estándar histórico de la industria broadcast para el transporte de vídeo sin comprimir sobre cable coaxial o fibra óptica [3]. Ofrece latencia sub-imagen y sincronismo garantizado, pero está limitado a distancias de hasta 100 m sin regeneradores y requiere infraestructura de alto coste.
- **SMPTE ST 2022-6 (SDI sobre IP)**: encapsula la señal SDI íntegra en paquetes UDP/IP, facilitando la migración de infraestructuras SDI existentes hacia redes Ethernet estándar [12]. Incorpora corrección de errores FEC para compensar pérdidas propias de las redes IP.
- **SMPTE ST 2110 (producción IP nativa)**: estándar publicado en 2017 que transporta vídeo, audio y metadatos como flujos IP independientes, proporcionando mayor flexibilidad de encaminamiento que SDI [13]. La sincronización entre flujos se consigue mediante PTP (IEEE 1588) [14], con precisión de microsegundos.
- **NDI (*Network Device Interface*)**: protocolo propietario de Vizrt/NewTek [15] que transporta vídeo con compresión ligera sobre redes LAN Ethernet con latencia de un fotograma. Ampliamente soportado por cámaras PTZ, mezcladores hardware y herramientas de software de producción.
- **SRT (*Secure Reliable Transport*)**: protocolo de código abierto desarrollado por Haivision con corrección de errores ARQ y cifrado AES-128/256 [16]. Permite transportar la señal desde el lugar del evento hasta el centro de producción remoto sobre Internet pública con latencia inferior a 1 s. FFmpeg lo soporta como entrada y salida, lo que permite integrarlo directamente en pipelines de Voctomix.
- **RTP (*Real-time Transport Protocol*)**: protocolo de transporte en tiempo real definido en la RFC 3550 [17], utilizado junto con RTSP para la distribución de flujos multimedia desde cámaras IP profesionales y en sistemas SMPTE 2110.
- **RTSP (*Real Time Streaming Protocol*)**: protocolo de señalización definido en la RFC 7826 [18] que se emplea para establecer y controlar sesiones de streaming multimedia. Actúa como mando a distancia del flujo RTP: permite iniciar, pausar y detener la transmisión de una fuente IP.

Tabla 2.4: Comparativa de protocolos de contribución audiovisual.

Protocolo	Medio	Velocidad máx.	Latencia	Uso típico
SDI (12G)	Coaxial/fibra	11,88 Gbit/s	Sub-imagen	Estudio broadcast
SMPTE ST 2022-6	Ethernet IP	3 Gbit/s	Sub-imagen	Migración SDI/IP
SMPTE ST 2110	Ethernet IP	Variable	Sub-imagen	Producción IP nativa
NDI	Ethernet LAN	~100 Mbit/s	1 fotograma	Software, PTZ
SRT	Internet/LAN	Variable	<1 s	Contribución REMI IP
RTP/RTSP	LAN/WAN	Variable	Sub-imagen	Cámaras IP, ST 2110

2.3.2. Formatos de contribución

El formato contenedor agrupa y sincroniza los flujos de vídeo y audio durante el transporte entre fuente y mezclador.

- **Matroska (MKV)** [19]: contenedor flexible y extensible basado en un formato binario abierto. Voctomix lo emplea como formato de transporte interno: las fuentes envían señal I420 y audio PCM S16LE a 48 kHz encapsulados en MKV sobre conexiones TCP, garantizando sincronía audio-vídeo y permitiendo multiplexar múltiples pistas sin *overhead* significativo.

2.3.3. Códecs de contribución

En la etapa de contribución se prioriza la calidad y la mínima degradación sobre la eficiencia de compresión. Los códecs utilizados son de alta calidad, generalmente intra-frame, para evitar la acumulación de artefactos en cadenas de producción con múltiples etapas de procesamiento.

- **Vídeo sin comprimir (RAW I420)**: el vídeo se transporta sin ningún tipo de compresión, preservando íntegramente toda la información de la señal original. El formato I420 (YUV 4:2:0 planar) organiza los datos en tres planos separados: el plano de luminancia Y a resolución completa y los dos planos de crominancia Cb y Cr a resolución reducida a la mitad en ambas dimensiones. Este submuestreo de crominancia aprovecha la menor sensibilidad del sistema visual humano a las variaciones de color, resultando en 1,5 B/píxel de promedio frente a los 3 B/píxel de RGB sin comprimir. El ancho de banda de un flujo I420 a 1080p25 es determinista e independiente del contenido visual:

$$622,1 \text{ Mbit/s} = \frac{1920 \times 1080 \times 1,5 \text{ B/píxel} \times 25 \text{ fps} \times 8}{10^6}$$

Voctomix emplea I420 en todo el procesamiento interno del motor GStreamer mediante cápsulas `video/x-raw,format=I420`, lo que elimina cualquier conversión de formato durante el procesamiento y garantiza que la latencia de mezcla dependa únicamente del tiempo de cálculo sobre los píxeles. Para preservar la calidad colorimétrica, todos los flujos deben declarar espacio BT.709 (`colorimetry=bt709`) y rango televisivo (`tv range`), requisito que voctocore verifica en el *handshake* inicial con cada fuente. El coste de este enfoque es que los 622 Mbit/s por fuente obligan

a que todo el tráfico de vídeo circule en bucle local, ya sea mediante la interfaz de loopback del servidor en instalación monopuesto o a través de redes Docker internas de alta velocidad en el despliegue contenerizado.

- **Audio PCM (*Pulse-Code Modulation*)**: el audio se transporta sin compresión en formato PCM S16LE (muestras enteras de 16 bits en orden *little-endian*) a 48 kHz estéreo. Voctomix utiliza este formato para todo el procesamiento interno de audio, encapsulado junto al vídeo I420 en el mismo contenedor MKV. La ausencia de compresión garantiza la máxima calidad y elimina cualquier latencia de codificación.
- **Apple ProRes** [20]: familia de códecs de alta calidad desarrollados por Apple para producción profesional. Utilizan exclusivamente compresión intra-frame, lo que minimiza la degradación acumulativa en múltiples etapas de procesamiento. ProRes 422 HQ es el estándar en producción televisiva para postproducción.
- **VC-3 / DNxHD (SMPTE ST 219)** [21, 22]: códec intra-frame estandarizado por SMPTE como VC-3 y distribuido comercialmente por Avid bajo el nombre DNxHD (y su variante de alta resolución DNxHR). Equivalente a ProRes en su propósito, está ampliamente implantado en flujos de trabajo de postproducción broadcast y en sistemas de edición Avid Media Composer.
- **JPEG 2000** [23]: códec que ofrece compresión sin pérdidas o con pérdidas mínimas y latencia muy reducida gracias a su arquitectura de codificación por bloques independientes. Utilizado en sistemas de contribución broadcast de alta calidad y en señalización digital.
- **JPEG XS (ISO/IEC 21122)** [24]: códec intra-frame de nueva generación publicado en 2019 específicamente diseñado para producción profesional en tiempo real. Su latencia de codificación es inferior a un milisegundo con compresiones de 4:1 a 6:1 sin pérdidas visualmente relevantes, lo que lo hace adecuado para contribución 4K y 8K en vivo. Ha sido adoptado en iniciativas de producción IP de alta calidad donde JPEG 2000 resulta excesivamente exigente computacionalmente.

Tabla 2.5: Comparativa de códecs de contribución audiovisual.

Códec	Compresión	Latencia enc.	Uso típico
RAW (I420/UYYVY)	Sin compresión	Nula	Vídeo interno Voctomix / estudio
PCM S16LE (audio)	Sin compresión	Nula	Audio interno Voctomix
Apple ProRes	4:1 – 8:1	<1 fotograma	Producción TV, postproducción
VC-3 / DNxHD	4:1 – 7:1	<1 fotograma	Broadcast, Avid workflows
JPEG 2000	4:1 – 20:1	<5 ms	Contribución broadcast
JPEG XS	4:1 – 6:1	<1 ms	Contribución 4K/8K en vivo

2.4. Distribución Audiovisual

Una vez generada la señal de programa por el mezclador, esta debe distribuirse hasta la audiencia. Esta etapa difiere fundamentalmente de la contribución: mientras que en la contribución se prioriza la calidad y la latencia mínima, en la distribución el objetivo

es llegar al máximo número de espectadores con los recursos de red disponibles, lo que obliga a comprimir la señal de forma agresiva y a equilibrar el compromiso entre latencia, calidad y escalabilidad.

2.4.1. Protocolos de distribución

Los protocolos de distribución gobiernan cómo la señal de programa viaja desde el mezclador hasta las plataformas de streaming y, desde ahí, hasta el reproductor del espectador.

- **RTMP (*Real-Time Messaging Protocol*)**: protocolo TCP de Adobe, estándar de ingesta para plataformas como YouTube Live o Twitch con latencia de 2–5 s. Aunque obsoleto en reproducción desde la desaparición de Flash, sigue siendo el protocolo de ingesta más soportado, con una adopción del 58 % en 2025 [25]. Voctomix lo soporta como salida mediante FFmpeg.
- **HLS (*HTTP Live Streaming*)**: protocolo de Apple que divide el vídeo en segmentos servidos mediante HTTP [26]. Latencia de 10–30 s en modo estándar, reducible a 2–5 s con *Low-Latency HLS*. Es el protocolo de reproducción más compatible, con soporte nativo en prácticamente todos los dispositivos y CDN.
- **MPEG-DASH (*Dynamic Adaptive Streaming over HTTP*)**: estándar abierto ISO/IEC 23009-1 [27] equivalente a HLS pero sin dependencia de proveedor. Divide el contenido en segmentos HTTP con ajuste adaptativo de calidad (ABR) y es empleado por YouTube, Netflix y Amazon Prime. Puede generarse con FFmpeg como alternativa abierta a HLS en el pipeline de Voctomix.
- **WebRTC (*Web Real-Time Communication*)**: estándar W3C para comunicación multimedia en tiempo real desde el navegador, con latencias de 100–500 ms. Su escalabilidad para audiencias masivas es limitada, pero resulta ideal para monitorización remota del sistema de producción y retornos entre el equipo técnico.

Tabla 2.6: Comparativa de protocolos de distribución audiovisual en directo.

Protocolo	Latencia	Transporte	Cifrado	Uso principal
RTMP	2–5 s	TCP	No (RTMPS sí)	Ingesta a plataformas
HLS	2–30 s	HTTP	HTTPS	Reproducción masiva (CDN)
MPEG-DASH	2–30 s	HTTP	HTTPS	Streaming adaptativo abierto
WebRTC	100–500 ms	UDP	DTLS/SRTP	Monitorización, retornos

2.4.2. Formatos de distribución

El formato contenedor de distribución define cómo se empaquetan los flujos de vídeo y audio codificados para su transporte hasta el reproductor final.

- **MPEG-TS (*Transport Stream, ISO 13818-1*)** [28]: estándar diseñado específicamente para transmisión broadcast con tolerancia a errores. Es el contenedor asociado a HLS, la emisión por satélite y la TDT. Su robustez frente a pérdidas de paquetes lo hace el formato de referencia para distribución broadcast.

- **MP4 / ISOBMFF (*ISO Base Media File Format*)** [29]: contenedor de referencia para distribución por HTTP progresivo y streaming adaptativo (MPEG-DASH, HLS fragmentado). Es el formato de almacenamiento estándar para grabaciones y distribución bajo demanda.
- **MOV (*QuickTime Movie*)** [30]: contenedor desarrollado por Apple, ampliamente utilizado en cámaras de producción profesional como formato de grabación nativo (Sony, Canon, Blackmagic). Comparte estructura con MP4/ISOBMFF al ser su antecedente directo. Es habitual en flujos de distribución hacia plataformas de edición y archivado.
- **FLV (*Flash Video*)**: contenedor legado asociado a RTMP. Aunque técnicamente obsoleto como formato de reproducción desde la retirada de Adobe Flash en 2020, sigue siendo el contenedor utilizado internamente en el protocolo RTMP para la ingesta hacia plataformas de streaming en directo.

2.4.3. Códecs de distribución

En la etapa de distribución se emplean códecs de alta eficiencia que comprimen la señal para reducir el ancho de banda necesario hasta el espectador final, asumiendo cierta pérdida de calidad a cambio de viabilizar el transporte.

- **H.264 / AVC (*Advanced Video Coding*)** [31]: el códec de vídeo más extendido en la actualidad y la línea de base de la industria del streaming. Introdujo herramientas avanzadas de predicción inter-fotograma (CABAC, transformada adaptativa $4 \times 4/8 \times 8$) que establecieron en 2003 el estándar de eficiencia vigente durante dos décadas. H.264 se organiza en perfiles que determinan el conjunto de herramientas de codificación disponibles; en producción audiovisual se emplea principalmente el perfil *High*, que activa CABAC y transformadas 8×8 y es soportado por la totalidad de plataformas CDN y decodificadores hardware desde 2010.

Voctomix emplea H.264 como códec de salida por defecto a través de FFmpeg y la biblioteca de software libre `libx264`. La señal de programa RAW I420 que emite voctocore es consumida directamente por FFmpeg para codificación en tiempo real, sin etapa de conversión de formato intermedia. La salida se encapsula en MPEG-TS para ingesta mediante RTMP o se segmenta en HLS para distribución adaptativa. La universalidad del perfil *High* garantiza compatibilidad con YouTube Live, Twitch y cualquier CDN comercial sin configuración adicional, mientras que la velocidad de codificación de `libx264` permite realizar la codificación en tiempo real sin aceleración hardware en CPU de gama media.

- **H.265 / HEVC (*High Efficiency Video Coding*)**: sucesor de H.264 que aproximadamente duplica la eficiencia de compresión, permitiendo la misma calidad perceptual a la mitad del bitrate [32]. Su codificación es 5–8 veces más lenta que H.264, lo que lo hace adecuado para streaming en tiempo real cuando se dispone de aceleración hardware (GPU o ASIC).
- **AV1**: códec abierto desarrollado por la Alliance for Open Media que mejora la eficiencia de H.265 en aproximadamente un 50 % adicional [33]. Su limitación en producción en directo es la velocidad de codificación: AV1 tarda entre 5 y 10 veces más que H.265, lo que restringe su uso en tiempo real a plataformas con aceleración

hardware dedicada. YouTube, Netflix y Facebook ya codifican gran parte de su contenido en AV1 para streaming bajo demanda.

- **AAC / Opus:** códecs de audio utilizados habitualmente junto a los anteriores. AAC (*Advanced Audio Coding*) es el estándar en distribución broadcast y plataformas de streaming; Opus es el códec preferido en aplicaciones WebRTC por su eficiencia a bajas tasas de bits.

Tabla 2.7: Comparativa de códecs de vídeo para distribución en directo.

Códec	Eficiencia vs H.264	Vel. encoding	Compatibilidad	RT posible
H.264/AVC	Base	Muy rápido	Universal	Sí
H.265/HEVC	×2	5–8× más lento	Alta (post 2015)	Sí (con HW)
AV1	×3	5–10× más lento	Media (creciendo)	Solo con HW

2.5. Casos de uso reales de producción remota

La producción remota ha dejado de ser una propuesta experimental para convertirse en el modelo operativo de referencia en eventos de gran escala. En esta sección se recogen cuatro casos de uso representativos que ilustran el rango de aplicaciones, desde eventos masivos con presupuesto de televisión pública hasta despliegues académicos de investigación.

2.5.1. Chaos Communication Congress (C3VOC)

El Chaos Communication Congress (CCC) es uno de los mayores congresos de tecnología y seguridad informática del mundo, celebrado anualmente en Alemania con más de 17 000 asistentes y múltiples salas en emisión simultánea. El equipo de producción de vídeo del congreso, C3VOC, desarrolló Voctomix [7] precisamente para cubrir esta necesidad: un mezclador software de código abierto capaz de gestionar varias salas en paralelo sin depender de hardware broadcast propietario.

Cada sala del congreso dispone de su propio servidor con una instancia de voctocore, alimentado por cámaras que envían señal RAW I420 sobre TCP/MKV y operado remotamente por un realizador a través de voctogui. Las grabaciones finales y la emisión en directo se publican en el portal de media del congreso (media.ccc.de). Este caso de uso es el origen y banco de pruebas principal del proyecto: Voctomix 2.0 hereda la arquitectura cliente-servidor de voctocore originalmente diseñada para este entorno.

2.5.2. Juegos Olímpicos de Tokio 2020 (OBS)

Los Juegos Olímpicos de Tokio 2020, celebrados en 2021, constituyeron el mayor despliegue de producción remota en la historia del olimpismo. Olympic Broadcasting Services (OBS) implementó un modelo REMI a gran escala en el que la señal de las cámaras de todas las sedes olímpicas se transportó a un centro de producción centralizado en Tokio, reduciendo drásticamente el número de equipos y personal técnico desplazado a cada instalación [34].

Este despliegue validó la viabilidad técnica del modelo REMI para eventos con decenas de sedes simultáneas, señales de alta resolución y requisitos de fiabilidad broadcast. La

reducción de la huella de carbono asociada al transporte de equipos fue uno de los argumentos adicionales que aceleraron la adopción del modelo remoto como estándar para ediciones futuras.

2.5.3. UEFA Champions League (Telefónica Servicios Audiovisuales)

Telefónica Servicios Audiovisuales implementó producción REMI para la retransmisión de partidos de la UEFA Champions League jugados en el estadio Santiago Bernabéu de Madrid [35]. Las cámaras del estadio enviaban la señal mediante SRT sobre Internet pública hasta el centro de producción remoto, donde los realizadores operaban el sistema como si estuvieran físicamente presentes en el estadio.

Este caso de uso demuestra la madurez del protocolo SRT para contribución broadcast en condiciones de red reales: el cifrado AES-256 garantiza la seguridad del contenido deportivo durante el transporte, y la corrección ARQ recupera los paquetes perdidos sin impacto perceptible en la latencia. La operación resultó transparente para el equipo de realización, que mantuvo su flujo de trabajo habitual sin adaptaciones significativas.

2.5.4. Proyecto CyberNEMO (Universidad Politécnica de Madrid)

En el marco del proyecto europeo de investigación CyberNEMO de la UPM, Voctomix 2.0 fue desplegado en un clúster Kubernetes de producción para validar la arquitectura de telemetría en condiciones reales. El broker RabbitMQ del sistema fue integrado con las colas del clúster (`vproduction-uc-media` y `vqprobe-uc-media`), y los eventos de mezclador, cambios de fuente activa, activaciones del stream blanker y métricas de rendimiento del servidor se publicaron en tiempo real durante sesiones de producción audiovisual técnica.

Este despliegue fue la primera validación del sistema en un entorno académico real, y demostró que Voctomix 2.0 puede integrarse con plataformas externas de monitorización sin necesidad de modificar el núcleo de procesamiento multimedia.

Tabla 2.8: Casos de uso representativos de producción remota audiovisual.

Caso de uso	Escala	Tecnología clave	Aportación
CCC / C3VOC	Múltiples salas	Voctomix, MKV/TCP	Origen del proyecto
Tokio 2020 (OBS)	Decenas de sedes	REMI cloud	Validación olímpica
UEFA CL (Telefónica)	Estadio único	SRT + AES-256	REMI sobre Internet
CyberNEMO (UPM)	Clúster K8s	RabbitMQ, telemetría	Validación académica

3. Diseño e implementación del sistema de producción remota de vídeo

Este capítulo describe el diseño e implementación de Voctomix 2.0: la arquitectura del sistema, sus componentes principales y las decisiones técnicas adoptadas durante el desarrollo. Para cada módulo se exponen los retos encontrados y los mecanismos implementados para resolverlos.

El trabajo comenzó con una fase de exploración sobre Voctomix 1.0 para familiarizarse con la herramienta antes de abordar la versión más reciente. Durante esas primeras semanas se estudió en detalle el fichero de configuración `default.ini`, se comprendió el funcionamiento del protocolo de puertos TCP de entrada de las cámaras (10000, 10001, 10002), se analizaron los flujos de previsualización de la interfaz gráfica y se exploró el funcionamiento de los composites. Esta inmersión en la versión anterior resultó clave para entender los fundamentos del pipeline GStreamer, la comunicación cliente-servidor, y la arquitectura del sistema, y sirvió de base sólida para dar el salto a Voctomix 2.0.

La figura 3.1 ofrece una visión global del sistema completo: las fuentes de señal, el núcleo de procesamiento, las salidas, la interfaz de control, la capa de telemetría, etc. Este diagrama sirve de hilo conductor para los apartados que siguen, cada uno de los cuales profundiza en uno de sus bloques.

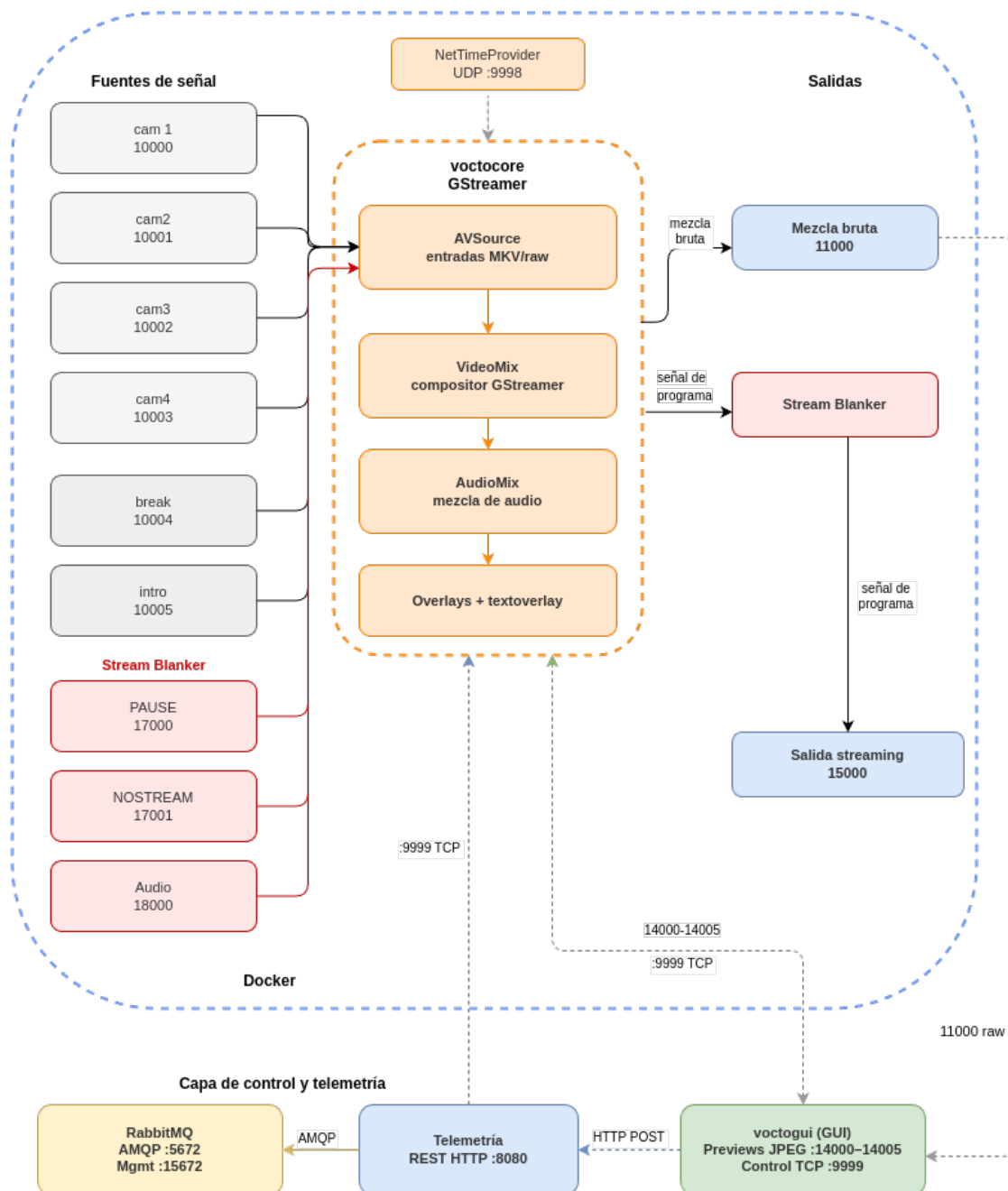


Figura 3.1: Arquitectura global del sistema Voctomix 2.0.

La tabla 3.1 recoge los puertos TCP/UDP del sistema y su función.

Tabla 3.1: Puertos del sistema Voctomix 2.0.

Puerto	Protocolo	Función
9998	UDP	Sincronización de reloj de red GStreamer
9999	TCP	Protocolo de control (GUI, scripts, telemetría)
10000	TCP	Entrada cámara 1 (MKV/raw)
10001	TCP	Entrada cámara 2 (MKV/raw)
10002	TCP	Entrada cámara 3 (MKV/raw)
10003	TCP	Entrada cámara 4 (MKV/raw)
10004	TCP	Entrada fuente break (vídeo de pausa)
10005	TCP	Entrada fuente intro (vídeo introductorio)
11000	TCP	Salida mezcla principal en bruto
12000	TCP	Salida principal de previsualización (GUI)
14000–14005	TCP	Salidas de previsualización comprimidas (GUI)
15000	TCP	Salida de streaming
17000	TCP	Entrada vídeo stream blanker PAUSE
17001	TCP	Entrada vídeo stream blanker NOSTREAM
18000	TCP	Entrada audio (música de fondo)
8080	HTTP	API REST del servicio de telemetría

3.1. Arquitectura del sistema

3.1.1. voctocore: núcleo multimedia

voctocore es el servidor central del sistema: gestiona todo el procesamiento multimedia, acepta las fuentes de vídeo entrantes, realiza la composición y expone los resultados en distintos puertos TCP. Se inicializa mediante `voctocore/voctocore.py`, que instancia en orden el **Pipeline** central (grafo GStreamer), el **ControlServer** TCP y el proveedor de tiempo de red GStreamer (**NetTimeProvider**, puerto UDP 9998). El **Pipeline** construye dinámicamente la cadena GStreamer a partir de la configuración INI: crea una instancia de **AVSource** por cada fuente declarada, instancia el **VideoMix** y el **AudioMix**, conecta el **StreamBlanker** y abre los puertos de salida.

El **ControlServer** expone el **protocolo de control** en el puerto TCP 9999: comandos de texto plano del tipo `set_video_a cam2` o `set_composite_mode pip`. Cada cambio de estado se difunde por *broadcast* a todos los clientes conectados, de forma que la interfaz gráfica, los scripts de automatización y el servicio de telemetría mantienen una vista coherente del sistema en todo momento.

3.1.2. voctogui: interfaz gráfica de control

voctogui es el cliente de control: implementado en PyGTK3, proporciona al realizador la interfaz visual para operar el sistema en tiempo real. Se conecta al núcleo mediante

un socket TCP no bloqueante gestionado con `GObject.io_add_watch`, de forma que los eventos de red no bloquean el hilo principal de GTK. La GUI recibe los flujos de pre-visualización de cada fuente en formato JPEG comprimido (puertos 14000–14005) para minimizar el consumo de ancho de banda, mientras que el monitor principal recibe la mezcla final en bruto (puerto 11000).

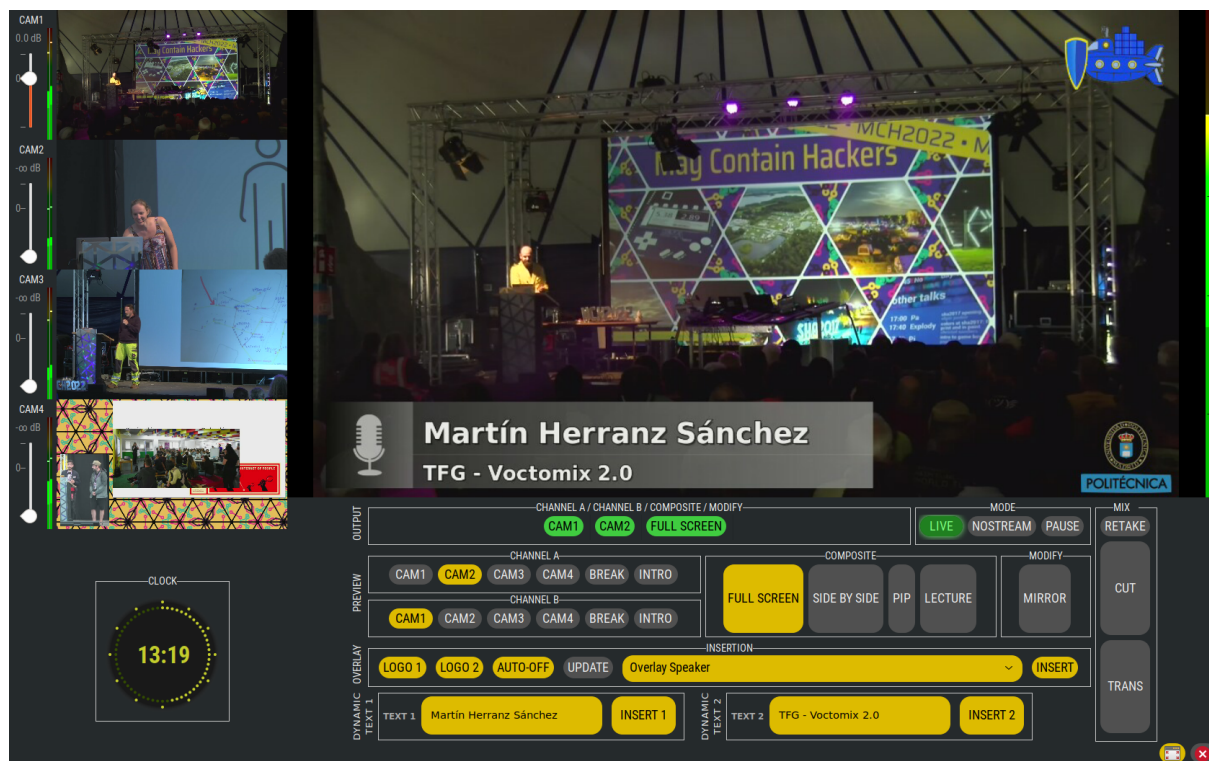


Figura 3.2: Interfaz gráfica completa de Voctomix 2.0 (voctogui).

3.2. Fuentes de señal de cámaras

El sistema acepta hasta cuatro fuentes de cámara simultáneas (cam1–cam4), que constituyen las entradas principales del mezclador. Cada cámara envía su señal al núcleo mediante una conexión TCP en formato Matroska con vídeo en bruto (I420) y audio PCM S16LE a 48 kHz estéreo.

3.2.1. Conexión de las fuentes

Cada fuente de cámara se conecta al puerto TCP correspondiente de voctocore:

- **cam1** → puerto 10000
- **cam2** → puerto 10001
- **cam3** → puerto 10002
- **cam4** → puerto 10003

El envío de la señal puede realizarse con cualquier herramienta capaz de producir un flujo Matroska sobre TCP. En el entorno de desarrollo y pruebas, las fuentes de cámara se

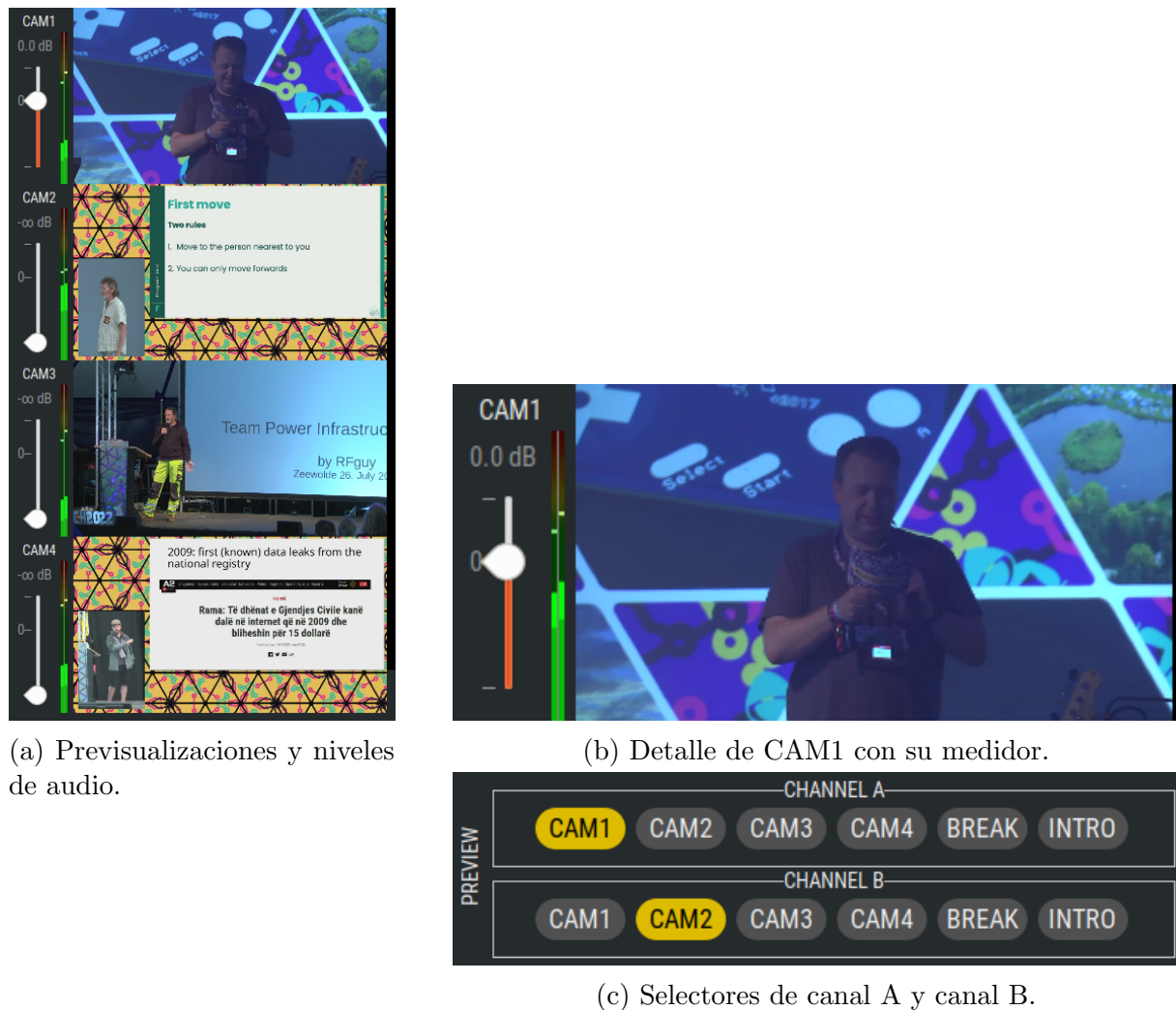
simulan mediante vídeos pregrabados de un evento real: grabaciones de distintas cámaras colocadas en diferentes puntos de un auditorio durante unas conferencias o charlas técnicas, que reproducen en bucle para emular la presencia de cámaras en directo. En un despliegue con cámaras físicas, la señal se encapsularía en Matroska/TCP mediante FFmpeg y una tarjeta de captura conectada al equipo servidor.

3.2.2. Previsualización en la interfaz gráfica

Una vez conectadas, las fuentes de cámara aparecen automáticamente en el panel de previsualizaciones de vooctogui. La interfaz muestra cuatro miniaturas JPEG de las fuentes activas, recibidas desde los puertos 14000–14003 a una resolución reducida (1024×576) para minimizar el consumo de ancho de banda entre el servidor y el cliente de control. Cada miniatura incluye:

- Un indicador visual del nombre de la fuente (cam1, cam2, cam3, cam4).
- Botones de selección para asignar la fuente al bus A o al bus B del mezclador.
- Un indicador del nivel de audio asociado a esa fuente.

El realizador selecciona la fuente activa haciendo clic sobre la miniatura correspondiente, lo que envía el comando `set_video_a <fuente>` o `set_video_b <fuente>` al núcleo mediante el protocolo de control TCP.



(a) Previsualizaciones y niveles de audio.

(b) Detalle de CAM1 con su medidor.

(c) Selectores de canal A y canal B.

Figura 3.3: Panel de cámaras en vodcast.

3.2.3. Requisitos de formato de la señal de entrada

Durante el desarrollo se comprobó que vodcast es estricto con los metadatos del flujo de vídeo entrante. Al intentar inyectar un archivo `.mp4` convencional como fuente de prueba, el núcleo rechazaba la conexión sin emitir un mensaje de error claro. Tras el diagnóstico se identificaron dos requisitos obligatorios que toda fuente debe cumplir:

1. **Pista de audio obligatoria:** vodcast exige que cada flujo Matroska contenga tanto vídeo como audio. Los vídeos sin audio deben complementarse con una pista de silencio generada en tiempo real por FFmpeg mediante el filtro `-f lavfi -i aevalsrc=0`. Sin este ajuste, el motor GStreamer cierra la conexión en el *handshake* inicial.
2. **Colorimetría televisiva (BT.709):** el núcleo exige que los metadatos del flujo declaren el espacio de color estándar de alta definición (`colorimetry=bt709`) y el rango de luz televisivo (`tv range`). Los archivos grabados con cámaras convencionales o editados con software de consumo suelen emplear el rango `pc` (full range). Para convertirlos, se aplican los filtros FFmpeg `scale=out_range=tv` y `colorspace=bt709`, que recalculan los valores de píxel en tiempo real sin degradación visual perceptible.

Estos requisitos deben tenerse en cuenta al integrar cualquier fuente de vídeo externa, ya sea procedente de cámaras físicas, archivos grabados o generadores de señal de prueba.

3.3. Fuentes de señal auxiliares

Además de las cuatro fuentes de cámara, el sistema incorpora varias fuentes de señal auxiliares que permiten gestionar el estado de la emisión en momentos en los que la mezcla principal no debe ser visible para el espectador, así como una entrada de audio independiente para música de fondo.

3.3.1. Fuente **break** e **intro**

break e **intro** son fuentes de vídeo pregrabadas que el núcleo trata exactamente igual que una cámara adicional: se conectan a los puertos 10004 y 10005 respectivamente mediante procesos FFmpeg, y aparecen en el mezclador como dos entradas más. Sin embargo, dado que no aportan información relevante al realizador durante la producción en directo, se ha optado por ocultarlas en la interfaz gráfica, manteniendo su funcionalidad completa en el núcleo sin ocupar espacio en el panel de previsualizaciones.

- **break**: vídeo en bucle empleado como pantalla de pausa animada durante los descansos del evento. FFmpeg lo envía con la opción `-stream_loop -1` para que la fuente nunca quede en negro, independientemente de la duración del archivo. El realizador puede seleccionarlo directamente como fuente activa del mezclador, igual que las cámaras 1 a 4.
- **intro**: vídeo introductorio que se reproduce al inicio del evento o entre sesiones. A diferencia del **break**, la **intro** *no* se reproduce en bucle continuo, sino que se dispara una sola vez cuando el realizador selecciona esa fuente en la interfaz gráfica.

Mecanismo de disparo de la **intro** (*vigilante*)

El comportamiento de disparo único de la **intro** plantea un reto arquitectónico: la GUI de **votogui** no puede ejecutar comandos externos sin bloquear el hilo principal de GTK. Para resolverlo sin modificar el núcleo del sistema, se implementó un proceso independiente denominado `vigilante_intro.py`.

Este script actúa como un cliente silencioso del puerto de control TCP (9999): escucha el flujo de notificaciones de estado que **votocore** difunde por *broadcast* y, cuando detecta el mensaje `video_status intro` (el realizador ha seleccionado la fuente **intro**), ejecuta inmediatamente el script `lanzar_intro.sh` mediante `subprocess.Popen` sin buffering para garantizar una reacción instantánea.

Esta arquitectura sigue la filosofía de separación de responsabilidades de **Voctomix**: el núcleo solo sabe mezclar señales; los automatismos de producción se implementan como clientes externos que reaccionan a su protocolo de estado. Si el **vigilante** fallara, el sistema de mezcla continuaría operando con plena normalidad.

3.3.2. Entrada de audio

El sistema incorpora una entrada de audio independiente en el puerto 18000, pensada para inyectar música de fondo durante los estados de pausa o fuera de emisión. Esta entrada acepta un flujo Matroska de sólo audio (PCM S16LE, 48 kHz, estéreo) y su nivel de salida puede controlarse desde el protocolo de control.

3.3.3. Stream Blanker: estados de emisión

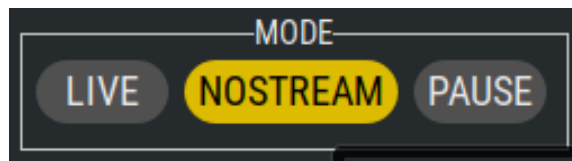
El stream blanker es el mecanismo que permite interrumpir la señal de programa hacia el exterior sin interrumpir el flujo de trabajo interno del realizador. Se implementa en `votocore/lib/streamblanker.py` como un elemento GStreamer adicional interpuesto entre la mezcla y el puerto de salida de streaming (puerto 15000).

El sistema define tres estados de emisión, controlables desde la barra de herramientas de votogui:

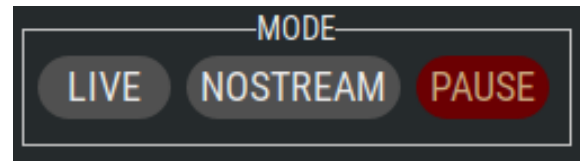
- **LIVE:** se emite la mezcla activa producida por votocore. El realizador controla en todo momento lo que ve el espectador.
- **PAUSE:** se emite el vídeo de pausa procedente del puerto 17000, acompañado de la música de fondo del puerto 18000. Se usa durante los descansos del evento para que los espectadores vean una pantalla animada mientras el equipo de producción prepara la siguiente sesión.
- **NOSTREAM:** se emite un vídeo de “fuera de emisión” procedente del puerto 17001, indicando al espectador que la transmisión ha finalizado o aún no ha comenzado.

El cambio entre estados se aplica sobre la señal de salida. Desde la interfaz gráfica, el realizador dispone de tres botones (LIVE, PAUSE, NOSTREAM) en la barra de herramientas.

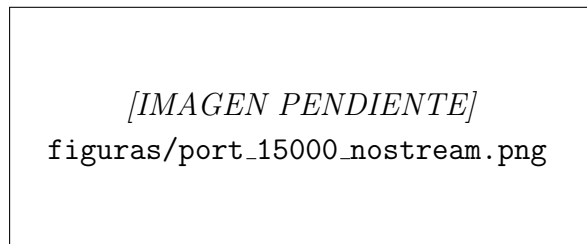
La figura 3.4 muestra la interfaz del realizador y la señal vista por el espectador en los estados NOSTREAM y PAUSE.



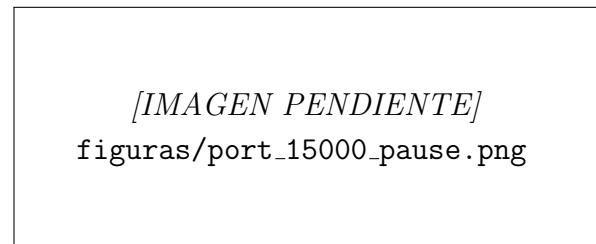
(a) GUI en estado NOSTREAM.



(c) GUI en estado PAUSE.



(b) Señal del espectador en NOSTREAM.



(d) Señal del espectador en PAUSE.

Figura 3.4: Estados NOSTREAM y PAUSE del stream blanker: vista del realizador (arriba) y señal recibida por el espectador (abajo).

3.4. Composites

Los composites definen cómo se combinan visualmente las dos fuentes de vídeo activas (bus A y bus B) en el fotograma de salida. Voctomix implementa los composites mediante el elemento GStreamer `compositor`, que sitúa y escala cada fuente de acuerdo con los parámetros geométricos definidos en la sección `[composites]` del fichero de configuración INI. La modificación de estos parámetros en tiempo real permite transiciones suaves entre composites sin generar fotogramas corruptos.

Los composites disponibles en Voctomix 2.0 son los siguientes:

La figura 3.5 ilustra la disposición visual de las dos fuentes (A en naranja, B en azul) en cada uno de los composites principales.

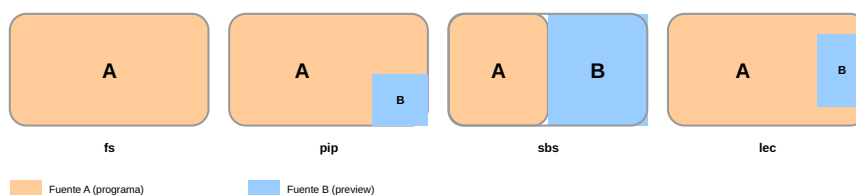


Figura 3.5: Disposición visual de las fuentes A y B en los cuatro composites principales de Voctomix 2.0: pantalla completa (**fs**), imagen en imagen (**pip**), lado a lado (**sbs**) y conferencia (**lec**).

3.4.1. Full Screen (fs)

La fuente A ocupa toda la pantalla y la fuente B queda oculta ($\alpha = 0$). Es el modo por defecto de una realización convencional: el realizador selecciona una fuente y la emite a pantalla completa. Resulta adecuado para la emisión de una ponencia individual, una demostración en pantalla o cualquier situación en la que una única fuente deba ocupar todo el encuadre.

3.4.2. Picture in Picture (pip)

La fuente A se muestra a pantalla completa mientras la fuente B aparece reducida en una esquina del encuadre (típicamente la esquina inferior derecha), a una escala aproximada del 27 % del ancho del fotograma. Es el composite clásico de “ponente con presentación”: la cámara del ponente ocupa la pantalla completa y las diapositivas aparecen en la esquina, o viceversa. Puede configurarse con `mirror=true` para que la fuente A se invierta horizontalmente, útil cuando la cámara tiene la imagen especular por la disposición del equipo.

3.4.3. Side by Side (sbs)

Ambas fuentes se muestran a igual tamaño, situadas una junto a la otra y ocupando cada una aproximadamente la mitad del fotograma. Es útil para comparaciones visuales, debates con dos participantes o para mostrar simultáneamente la cámara del ponente y sus diapositivas con el mismo peso visual.

3.4.4. Lecture (lec)

La fuente A ocupa la mayor parte del encuadre (aproximadamente el 75 %) mientras que la fuente B aparece reducida en la parte derecha. Este composite está específicamente diseñado para retransmisiones de ponencias o clases: las diapositivas (fuente A) dominan el encuadre y la cámara del ponente (fuente B) aparece en un recuadro lateral de menor tamaño, manteniendo siempre la legibilidad del contenido proyectado.

La variante `lec_43` está concebida para acomodar contenido con relación de aspecto 4:3 (como proyectores antiguos o capturas de escritorio con resolución 1024×768) en un encuadre 16:9. En la versión original del repositorio, esta variante aplicaba un recorte del 31 % mediante el parámetro `crop-a=0.125/0` que destruía el encuadre completo cuando la fuente era nativa 16:9. Se corrigió redefiniendo los parámetros de posición y escala en la sección `[composites]` del fichero INI para eliminar el recorte y ajustar la ventana secundaria de forma que encaje perfectamente a la derecha sin desbordarse del fotograma.

3.4.5. Mirror

Aunque no es un composite independiente sino una propiedad disponible en varios de los anteriores (`mirror=true` en `pip`, `lec` y `fs-pip`), el efecto espejo permite invertir horizontalmente la fuente A o B en el composite. Esto es necesario cuando la señal de cámara llega especularmente invertida, como puede ocurrir con ciertos modelos de cámaras web o cuando la cámara está orientada de forma atípica. Al activar el `mirror` desde el

fichero de configuración, el núcleo aplica la inversión directamente en el compositor de GStreamer sin coste adicional.

[IMAGEN PENDIENTE: captura del selector de composites en voctogui, mostrando los botones FULL SCREEN, PIP, SIDE BY SIDE y LECTURE] con el composite activo resaltado.
*Guardar como **figuras/panel_composites.png** y enviar.*

Figura 3.6: Selector de modos de composite en la interfaz de Voctomix 2.0.

3.5. Transiciones

Las transiciones controlan cómo se realiza el cambio de una fuente o composite a otro en el flujo de programa. Voctomix 2.0 ofrece tres modos de transición que el realizador activa desde la barra de herramientas de la interfaz gráfica.

3.5.1. Cut (corte instantáneo)

El botón **Cut** realiza una transición instantánea: en el siguiente fotograma tras la pulsación, la fuente de programa pasa a ser la fuente de preview. No existe ningún efecto intermedio. Es la transición estándar en producción broadcast: el corte imita el montaje cinematográfico y resulta imperceptible cuando el *timing* es correcto.

3.5.2. Trans (transición suave)

El botón **Trans** realiza una transición animada entre el estado actual y el nuevo estado de composite o fuente, interpolando los parámetros geométricos y de transparencia durante un tiempo configurable (por defecto 750 ms, definido en la sección [transitions] del fichero INI). La transición se implementa modificando progresivamente los valores de los pads del compositor de GStreamer.

Voctomix 2.0 define múltiples trayectorias de transición según el estado de origen y destino (por ejemplo, de **fs** a **pip** la transición pasa por el composite intermedio **fs-pip** antes de llegar a **pip**), garantizando que la animación sea visualmente coherente independientemente del cambio solicitado.

3.5.3. Retake

El botón **Retake** permite al realizador deshacer el último cambio de fuente o composite y volver al estado inmediatamente anterior sin interrumpir la señal de programa. Su funcionamiento es equivalente a un Cut inverso: en el siguiente fotograma, el sistema intercambia programa y preview, restaurando la fuente que estaba en antena antes del cambio. Esta

función resulta especialmente útil cuando el realizador realiza un corte erróneo y necesita corregirlo de inmediato sin generar un segundo corte visible en la emisión.

*[IMAGEN PENDIENTE: captura de los botones CUT,
TRANS y RETAKE
en la barra de herramientas de voctogui.
Guardar como **figuras/panel_transiciones.png** y enviar.*

Figura 3.7: Botones de transición en la interfaz de Voctomix 2.0: CUT, TRANS y RETAKE.

3.6. Overlays, logos y texto dinámico

Voctomix 2.0 incorpora un sistema completo de superposición de elementos gráficos sobre la señal de vídeo: logos corporativos, overlays PNG con canal alfa y texto dinámico renderizado en tiempo real. Todos estos elementos se gestionan desde la interfaz gráfica sin interrumpir el pipeline de procesamiento.

3.6.1. Logos

El sistema soporta hasta dos logos simultáneos, superpuestos en posiciones fijas del encuadre definidas mediante la configuración INI:

- **logo1**: posicionado en la esquina inferior derecha del fotograma.
- **logo2**: posicionado en la esquina superior izquierda del fotograma.

Los ficheros PNG de los logos se definen en las secciones `[logo1]` y `[logo2]` del fichero de configuración, permitiendo adaptar la identidad visual del evento sin modificar el código. La visibilidad de cada logo se puede activar o desactivar individualmente desde la interfaz gráfica.

3.6.2. Overlays PNG y texto dinámico

Los overlays son imágenes PNG con canal alfa que se superponen sobre la señal de vídeo mediante el elemento GStreamer `gdkpixbufoverlay`. El módulo `voctocore/lib/overlay.py` gestiona el ciclo de vida completo de un overlay: carga del fichero PNG, fundido de entrada (*blend-in*), mantenimiento en pantalla y fundido de salida (*blend-out*). El tiempo de fundido se controla mediante el parámetro `blend-time` en la sección `[overlay]` del fichero INI (en milisegundos).

Durante el desarrollo se identificó una limitación del elemento `gdkpixbufoverlay` de GStreamer: está diseñado para leer archivos de imagen estáticos desde una ruta del sistema de ficheros, sin soporte nativo para actualizaciones en tiempo real. La solución adoptada

consiste en regenerar el fichero PNG en disco con el nuevo texto renderizado mediante Cairo y forzar la recarga del elemento GStreamer mediante el comando `set_overlay` del protocolo de control. Este mecanismo permite actualizar los rótulos en caliente durante la emisión con una latencia inferior a un fotograma.

3.6.3. Rotulación dinámica de ponente

El sistema de rotulación permite al operador escribir el nombre del ponente o el título de la sesión en un campo de texto de la interfaz gráfica y enviarlo al núcleo para su superposición inmediata sobre el vídeo. La cadena de procesamiento completa es la siguiente:

1. El operador escribe el texto en el campo `GtkEntry` del panel de inserciones de `voctogui`.
2. Al pulsar *Enter* o el botón **INSERT**, el módulo `overlay.py` de la GUI solicita a Cairo la renderización del texto sobre la plantilla PNG activa.
3. La imagen resultante se escribe en disco y se envía al núcleo mediante el comando `TCP set_overlay <ruta_fichero>`.
4. `votocore` recarga `gdkpixbufoverlay` con la nueva imagen, que aparece en antena con el efecto de fundido configurado.

3.6.4. Sistema de doble rotulación

Para aumentar la versatilidad del sistema de grafismo, se implementó un segundo motor de rotulación independiente que permite mostrar simultáneamente dos textos en posiciones distintas del encuadre. Los cambios realizados abarcan las cuatro capas del sistema:

- **Motor de vídeo** (`videomix.py`): se inyectó un segundo nodo `textoverlay` en el pipeline GStreamer (`rotulo_dinamico_2`), configurado con alineación horizontal opuesta al primero para evitar solapamientos. Se implementó el método `set_dynamic_text_2` para actualizar sus propiedades en caliente.
- **Protocolo de control** (`commands.py`): se amplió el servidor TCP con el comando `set_text2` y se configuró la notificación `text2_updated` para que cualquier cliente conectado sea informado del cambio.
- **Interfaz gráfica** (`votogui.ui` y `votogui.css`): se creó un segundo contenedor `GtkBox` en el panel de inserciones, situado junto al primero. Los nuevos selectores CSS (`#tfg-entry-2`, `#tfg-btn-2`) heredan la estética corporativa oscura y el efecto de iluminación amarilla al activarse.
- **Lógica de foco y teclado** (`overlay.py`): se perfeccionó el algoritmo de intercepción de atajos de teclado a nivel de ventana. El sistema evalúa dinámicamente qué campo de texto tiene el foco (`is_focus()`) para procesar correctamente las teclas críticas (Espacio, Retroceso, *Enter*), evitando colisiones con los atajos globales del mezclador.

3.6.5. Botón UPDATE: recarga dinámica de recursos

El botón **UPDATE** del panel de overlays permite recargar el directorio de recursos gráficos durante la emisión. Esta funcionalidad es crítica en entornos de directo: si el equipo de grafismo corrige un error ortográfico en una plantilla PNG o añade un nuevo rótulo de último minuto, el operador puede incorporarlo al sistema sin interrumpir la señal de vídeo ni reiniciar ningún proceso.

3.6.6. Auto-off de overlays

La funcionalidad *auto-off* resuelve un problema frecuente en producción en directo: el operador realiza un corte de cámara olvidando retirar el overlay activo, que queda superpuesto sobre la nueva fuente de forma inapropiada (por ejemplo, el nombre de un ponente apareciendo sobre la imagen de otro).

Cuando la opción `auto-off = true` está activa en la configuración, el módulo `overlay.py` registra un *listener* sobre el evento `video_status`. Cada vez que la fuente de vídeo activa cambia, el listener inicia automáticamente la secuencia de fundido de salida del overlay activo. Si hay texto dinámico superpuesto, éste se elimina junto al overlay gráfico, garantizando que ningún elemento de rotulación quede huérfano en pantalla.

El comportamiento puede configurarse mediante:

- `auto-off = true/false`: activa o desactiva la funcionalidad.
- `user-auto-off = true/false`: expone un botón de alternancia en la GUI para que el operador pueda activar/desactivar el auto-off en tiempo real.
- `blend-time = <ms>`: tiempo de fundido de salida en milisegundos.

[IMAGEN PENDIENTE: captura del panel de overlays en voctogui, mostrando los botones LOGO1, LOGO2, AUTO-OFF, UPDATE, los campos de texto dinámico (INSERT 1 e INSERT 2) y el botón de activación.

Guardar como `figuras/panel_overlays.png` y enviar.

Figura 3.8: Panel de overlays y rotulación dinámica en la interfaz de Voctomix 2.0.

3.7. Señal de programa final

Una vez procesada la mezcla — con el composite activo, las transiciones aplicadas y los overlays superpuestos — voctocore emite la señal de programa final de forma simultánea por dos puertos:

- **Puerto 11000**: salida de la mezcla en bruto (vídeo sin comprimir en formato I420), destinada a herramientas de grabación o encoders externos como FFmpeg. Esta

salida refleja siempre el estado real de la mezcla, independientemente del estado del stream blanker.

- **Puerto 15000:** salida de streaming, que pasa previamente por el stream blanker. Cuando el estado es LIVE, esta salida es idéntica al puerto 11000; cuando el estado es PAUSE o NOSTREAM, emite el vídeo alternativo correspondiente en lugar de la mezcla real.

El encoder de salida (típicamente FFmpeg) se conecta al puerto 15000 y produce el flujo H.264/AAC que se ingesta en la plataforma de distribución (RTMP/SRT hacia YouTube Live, Twitch u otras plataformas). La separación entre los puertos 11000 y 15000 garantiza que la grabación local refleje siempre la mezcla real del realizador, mientras que la audiencia remota solo recibe la señal que el operador autoriza explícitamente a través del stream blanker.

3.8. Personalización de la interfaz gráfica

Una vez que el pipeline de procesamiento multimedia estaba operativo y las funciones principales del sistema eran estables, se abordó la transformación visual y operativa de voctogui. La interfaz original presentaba un aspecto genérico, con fondo blanco heredado del tema del sistema operativo y sin iconografía coherente. Esta personalización se realizó de forma progresiva a lo largo del desarrollo, en paralelo a la incorporación de nuevas funcionalidades, con el objetivo de convertirla en una herramienta de producción profesional lista para su uso en eventos reales.

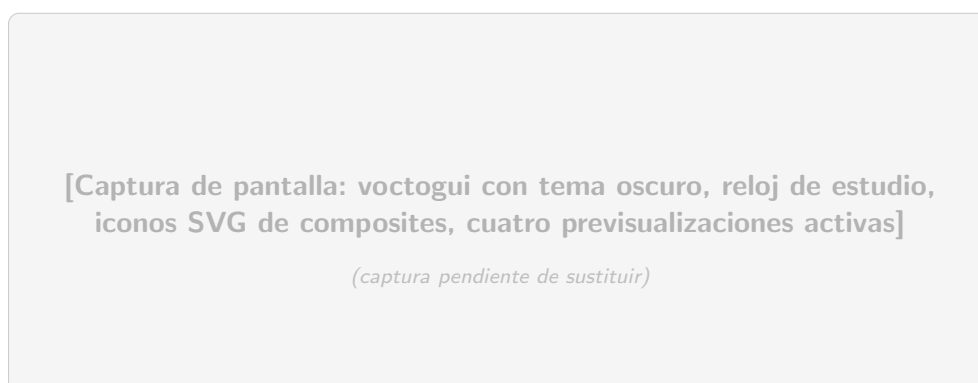


Figura 3.9: Interfaz gráfica voctogui tras la personalización: tema oscuro CSS (#252a2c), iconos SVG de composites, reloj de estudio y cuatro paneles de previsualización activos.

3.8.1. Tema oscuro y hoja de estilos

Se inyectó una hoja de estilos CSS (`voctogui.css`) que fuerza un fondo #252a2c y texto claro en todos los paneles. Los botones activos se iluminan en amarillo (#ddbb00), unificando el lenguaje visual de toda la interfaz y proporcionando al realizador una retroalimentación visual inmediata sobre el estado de cada control.

3.8.2. Iconografía SVG

Los botones de composite y de control de emisión se vincularon a iconos vectoriales `.svg`, declarados en las secciones `[toolbar.composites]` y `[toolbar.mix]` del fichero de configuración `default-config.ini`. El uso de SVG garantiza que los iconos se escalan sin pérdida de calidad a cualquier resolución de pantalla.

3.8.3. Reloj de estudio

Se integró el widget `StudioClock` en la parte inferior del panel de cámaras mediante el diseño XML de la interfaz (`voctogui.ui`), envuelto en un `GtkAspectFrame` para preservar las proporciones del panel de vídeo. El reloj muestra la hora del sistema en tiempo real, indispensable para la coordinación de escaleta en producciones en directo: el realizador puede sincronizar los cortes de cámara con los tiempos marcados en el guión sin apartar la vista del monitor de mezcla.

3.9. Sistema de telemetría

El sistema de telemetría de Voctomix 2.0 permite registrar en tiempo real todos los eventos que ocurren durante una sesión de producción: cambios de fuente activa, cambios de composite, activaciones del stream blanker, carga y descarga de overlays, niveles de audio, y métricas de rendimiento del servidor. Esta información resulta valiosa tanto para la monitorización durante el directo como para la auditoría posterior de la sesión. El módulo de todo lo que ocurre en la mesa de mezclas.

3.9.1. Arquitectura del servicio de telemetría

El servicio de telemetría (`example-scripts/ffmpeg/telemetry_service.py`) es un proceso Python independiente que no requiere privilegios especiales sobre el núcleo. Su arquitectura se articula en cinco capas funcionales:

Intercepción del núcleo por TCP

El servicio abre una conexión de socket TCP al puerto 9999 de voctocore, comportándose como un cliente de control más. Escucha en un hilo secundario (`threading`) los mensajes del protocolo interno, filtrando los eventos de estado relevantes: `video_status` (fuente en previo y en programa), `composite_mode` (distribución de pantalla activa), y `stream_status` (LIVE, PAUSE, NOSTREAM).

El handshake inicial incluye el comando `get_config`, que devuelve en un bloque JSON los parámetros del *pipeline*: resolución, *framerate* (frecuencia de imagen), formato de píxel (`yuv420p`), frecuencia de muestreo de audio y número de canales. Estos datos se inyectan en el campo `stream_info` de cada entrada del log, dotando al registro de validez técnica para análisis de rendimiento.

Extracción del estado de la interfaz gráfica

Se identificó un reto arquitectónico crítico: voctocore desconoce el estado de los elementos personalizados de la GUI (rótulos dinámicos, niveles de los *faders* de audio) porque estos

se gestionan puramente en la capa gráfica. Para capturarlos sin modificar el núcleo, se desarrolló el módulo `gui_state_exporter.py`, integrado en `votogui`.

Este módulo lee constantemente el estado de los widgets GTK mediante `Glib.idle_add` (para garantizar la seguridad de hilos) y exporta un fichero local `gui_state.json` con los niveles de audio por cámara (`cam1.db`, etc.) y el estado de la superposición de textos y logos. El servicio de telemetría consume este fichero JSON para completar la vista unificada del sistema.

Fusión de estados y doble lógica de registro

El servicio unifica los datos del núcleo (TCP) y de la interfaz (JSON local) mediante un sistema de fusión protegido por cerrojos de hilo (`threading.Lock()`) para evitar condiciones de carrera. El registro sigue una doble vía temporal:

- **Por evento (CHANGE):** en cuanto se detecta cualquier variación de estado, la entrada se escribe inmediatamente en el log con su marca de tiempo. Esta modalidad captura cada corte de cámara o cambio de composite en el instante exacto en que ocurre.
- **Por latido (STATE):** un hilo paralelo escribe el estado completo del sistema cada 5 segundos como medida de redundancia y control de *uptime*, independientemente de que haya habido cambios.

Telemetría técnica: métricas de rendimiento

Para dotar al log de utilidad operativa, el servicio registra en cada entrada un bloque `performance` con las siguientes métricas del servidor:

- **Ancho de banda por cámara (bandwidth_mbps):** el vídeo en bruto (Raw YUV420p) no varía con el contenido visual. El valor se calcula matemáticamente como $\text{Mbps} = \frac{\text{Ancho} \times \text{Alto} \times \text{FPS} \times 1,5}{1\,000\,000}$. Para 1080p25 el resultado es aproximadamente 622 Mbps por fuente, lo que evidencia que todo el tráfico de vídeo debe circular en bucle local (localhost o red interna Docker) sin atravesar interfaces físicas convencionales.
- **Uso de CPU (cpu_usage_percent):** leído de `/proc/stat`, permite al operador verificar que el servidor trabaja por debajo del umbral de saturación. Si la CPU supera el 80 %, GStreamer comienza a descartar fotogramas, degradando la emisión.
- **Memoria RAM (ram_usage_percent, ram_available_mb):** leído de `/proc/meminfo`. Un consumo bajo y estable confirma que los pipelines GStreamer procesan y liberan fotogramas en tiempo real sin acumular búferes en memoria.
- **Tiempos de sesión (uptime, session_duration):** `uptime` indica el tiempo de encendido del servidor; `session_duration` actúa como cronómetro del evento y permite correlacionar los cortes registrados en el JSON con el código de tiempo del archivo de grabación final.

Formato de salida: JSON Lines

Cada entrada del log se escribe como un objeto JSON completo en una línea del fichero `registroN.jsonl` (*JSON Lines*). Este formato permite leer el log de forma incremental

con herramientas estándar (jq, Python, scripts de análisis) sin necesidad de cargar el fichero completo en memoria, lo que resulta práctico en sesiones de larga duración.

Un ejemplo de entrada de tipo CHANGE tras un corte de cámara tendría la siguiente estructura:

```
{ "timestamp": "2025-03-28T20:14:32.105Z",  
  "type": "CHANGE",  
  "video": { "program": "cam2", "preview": "cam1" },  
  "composite": "fs",  
  "stream": "live",  
  "overlay": { "active": false, "text": "" },  
  "audio": { "cam1_db": -12.0, "cam2_db": 0.0 },  
  "performance": { "cpu": 34.2, "ram_pct": 8.7,  
                   "bandwidth_mbps": 622.1 },  
  "stream_info": { "width": 1920, "height": 1080,  
                  "fps": 25, "pix_fmt": "yuv420p" } }
```

Autodescubrimiento de red

Para que el log identifique el origen de los comandos en escenarios distribuidos (dos PCs, Docker), el servicio implementa una función de autodescubrimiento de red en dos niveles:

1. **Nivel contenedor (prioridad):** si existe la variable de entorno `MI_IP_REAL` (inyectada por Docker Compose), el servicio la utiliza directamente como IP de la interfaz LAN real del servidor.
2. **Nivel nativo (fallback):** si la variable no existe, el script activa un socket UDP hacia 8.8.8.8 sin llegar a realizar la conexión efectiva, interrogando a la tabla de enrutamiento del kernel para identificar qué interfaz física tiene salida a Internet y obteniendo así la IP real de la LAN de forma automática.

3.9.2. API REST y mensajería AMQP

El servicio expone los datos de telemetría en dos canales complementarios:

- **API REST HTTP (puerto 8080):** implementada con `http.server.HTTPServer` de la librería estándar de Python, sin dependencias externas. Devuelve el estado actual completo de la sesión en formato JSON. Para prevenir bloqueos de puerto en reinicios rápidos se activa la opción de socket `SO_REUSEADDR`, equivalente al parámetro `allow_reuse_address = True` de Python.
- **Mensajería AMQP (RabbitMQ):** cada cambio de estado se publica como un mensaje JSON en el exchange `vocotmix_events` del broker RabbitMQ, permitiendo que cualquier consumidor externo (sistema de streaming, dashboard web, automatización) se suscriba a los eventos sin acoplarse directamente al el servicio de telemetría ni a vocotcore.

3.9.3. RabbitMQ como broker de mensajería

RabbitMQ es el broker de mensajería AMQP que centraliza los eventos de telemetría. En el despliegue Docker se ejecuta como un contenedor independiente accesible en el puerto 5672 (protocolo AMQP) y 15672 (consola web de administración). Las credenciales por defecto son `vocmix/vocmix123`.

La integración con RabbitMQ permite:

- Mantener un registro persistente de todos los eventos de la sesión con marca de tiempo, para análisis posterior.
- Integrar Voctomix con sistemas externos de monitorización, *logging* centralizado (ELK Stack, Grafana) o automatización mediante consumidores de la cola.
- Verificar el correcto funcionamiento del sistema durante las pruebas, comprobando que cada acción del realizador genera el evento de telemetría correspondiente.

Un ejemplo de consumidor de la cola RabbitMQ se encuentra en `rabbit_consumer_example.py`, que sirve como punto de partida para integrar Voctomix con sistemas externos.

En el marco del proyecto europeo de investigación CyberNEMO (UPM), el broker RabbitMQ de Voctomix 2.0 fue integrado en el clúster Kubernetes de producción del proyecto, donde las colas `vproduction-uc-media` y `vqprobe-uc-media` recibieron los eventos del mezclador en tiempo real durante las pruebas reales de producción audiovisual. Este despliegue constituyó la primera validación del sistema de telemetría en un entorno de producción real, demostrando su capacidad de integración con plataformas externas de seguridad y monitorización.

```
~/voctomix
```

Para entrar desde la terminal

```
bash
```

```
cd ~/voctomix
```

Figura 3.10: Colas RabbitMQ activas durante la integración con CyberNEMO.

La arquitectura desacoplada del servicio de telemetría, conectado al núcleo como un cliente TCP ordinario sin privilegios especiales, garantiza que su eventual fallo no afecte al procesamiento multimedia principal: el sistema de producción opera con plenas garantías aunque el servicio de telemetría no esté disponible (*single point of failure* libre).

3.9.4. Evolución del protocolo de telemetría: de UDP a HTTP

En el escenario de dos PCs, el servicio de telemetría necesita recibir los datos de estado de la GUI (niveles de audio, estado de overlays) desde el PC del realizador a través de la red. La implementación inicial utilizó UDP (`socket.SOCK_DGRAM`), que presenta dos fallos críticos en entornos Wi-Fi:

- **Falta de fiabilidad:** UDP es *fire-and-forget*; los paquetes de estado pueden perderse sin que el emisor lo detecte, dejando el servicio y la GUI desincronizados.
- **Bloqueos del kernel (Errno 98):** al reiniciar los scripts rápidamente durante el desarrollo, el kernel mantenía el puerto UDP en estado `TIME_WAIT`, impidiendo que el servicio volviera a enlazar el puerto hasta que expirase el tiempo de gracia.

La solución fue migrar a un modelo **HTTP/TCP**: el servicio de telemetría actúa como servidor HTTP en el puerto 8080, y el `gui_state_exporter.py` del PC del realizador envía los datos de estado mediante peticiones HTTP POST con cuerpo JSON. TCP garantiza la entrega secuencial de los datos, y HTTP proporciona un estándar robusto para el intercambio de estructuras JSON. Adicionalmente, se estableció un *timeout* de 1 segundo en el cliente (`urlopen(req, timeout=1.0)`) para que la GUI nunca se bloquee si el PC del servidor no está disponible.

4. Despliegue del sistema

4.1. Scripts de arranque

Para simplificar el despliegue y evitar errores de configuración manual, se desarrollaron scripts de arranque que unifican en un único comando el lanzamiento de todos los componentes del sistema. Cada escenario de despliegue tiene su propio script.

4.1.1. Escenario monopuesto (`start_studio_single_pc.sh`)

Este script arranca el sistema completo en un único equipo: inicia voctocore, lanza el servicio de telemetría, inyecta las señales de continuidad (vídeo de pausa, vídeo *offline* y música de fondo), y finalmente abre voctogui. Antes de lanzar cada componente, un bucle comprueba con `ss -lnt` que el puerto 9999 está en escucha, evitando las condiciones de carrera que se producían con los `sleep` fijos del código original. Al cerrar voctogui, la instrucción `trap` captura la señal de salida y termina todos los procesos hijos de forma limpia.

4.1.2. Escenario dos PCs (`2pc_escenario2/`)

El escenario de dos equipos se gestiona con dos scripts independientes que deben ejecutarse en paralelo:

- `start_voctocore_pc1.sh` (PC servidor): arranca voctocore, las señales de continuidad y el servicio de telemetría en el equipo que actúa como servidor. Detecta automáticamente la IP de la interfaz LAN mediante la tabla de enrutamiento del kernel (`ip route get 1.1.1.1`), de modo que no es necesario configurar la IP a mano en cada despliegue.
- `start_voctogui_pc2.sh` (PC realizador): lanza voctogui y lo apunta a la IP del servidor mediante la variable de entorno `IP_SERVER`. Si no se especifica, el valor por defecto es `127.0.0.1` para pruebas en un solo equipo.

Ejemplo de uso desde el PC realizador:

```
IP_SERVER=192.168.1.50 ./2pc_escenario2/start_voctogui_pc2.sh
```

4.1.3. Escenario Docker (`launch_docker_studio.sh`)

Este script ejecuta `xhost +local:docker` para permitir que los contenedores accedan al servidor gráfico del anfitrión, exporta la IP local como variable de entorno (`HOST_IP`) para que el servicio de telemetría la incluya en los registros, y lanza el stack con `docker`

`compose up`. Al cerrar el terminal, la instrucción `trap` ejecuta `docker compose down` de forma automática.

El mismo flujo está disponible a través del Makefile:

```
make docker-up      # xhost + y docker compose up
make docker-down    # detener todos los servicios
make docker-restart # down + up en un solo paso
```

4.1.4. Escenario Kubernetes (launch_k8s.sh)

El script de Kubernetes comprueba si Minikube y el pod principal ya están en ejecución antes de lanzarlos, aplica los manifiestos con `kubectl apply`, espera a que todos los contenedores estén listos y establece un `kubectl port-forward` que expone los puertos del pod en `localhost` del anfitrión. Esto permite que `votogui` se conecte al clúster exactamente igual que en cualquier otro escenario. Al salir, un `trap EXIT` cierra el `port-forward` de forma limpia.

Tabla 4.1: Scripts de arranque por escenario de despliegue.

Escenario	Script	Comando de inicio
Un PC (nativo)	<code>start_studio_single_pc.sh</code>	<code>./start_studio_single_pc.sh</code>
Dos PCs (PC1)	<code>2pc_escenario2/start_votocore_pc1.sh</code>	<code>./start_votocore_pc1.sh</code>
Dos PCs (PC2)	<code>2pc_escenario2/start_votogui_pc2.sh</code>	<code>IP_SERVER=<IP>./start_votogui_pc2.sh</code>
Docker Compose	<code>launch_docker_studio.sh</code>	<code>make docker-up</code>
Kubernetes	<code>launch_k8s.sh</code>	<code>./launch_k8s.sh</code>

4.2. Despliegue con Docker Compose

El despliegue con Docker elimina la necesidad de instalar manualmente GStreamer, Python y sus dependencias en el equipo anfitrión. Todo el sistema se ejecuta dentro de contenedores reproducibles, de modo que el comportamiento es idéntico en cualquier máquina con Docker instalado. El despliegue se define en dos ficheros: `Dockerfile` (imagen base del sistema) y `docker-compose.yml` (conjunto de servicios).

El `Dockerfile` original del repositorio (año 2016) usaba Ubuntu 15.10 con dependencias ya obsoletas e imágenes de Docker Hub que ya no existen. Se reescribió desde cero sobre `ubuntu:22.04 LTS`.

4.2.1. Dockerfile

La imagen se construye sobre Ubuntu 22.04 LTS. Todas las dependencias se instalan en una sola capa para reducir el tamaño final:

- GStreamer 1.0 con sus plugins *base*, *good*, *bad* y *ugly*.
- FFmpeg, para la inyección de fuentes de vídeo.

- *Bindings* Python de GObject (`python3-gi`), la biblioteca Cairo para renderizado de *overlays*, y la librería AMQP (`pika`) para la integración con RabbitMQ.
- Utilidades de red (`netcat`, `iproute2`) para el *healthcheck* y el autodescubrimiento de IP.
- Bibliotecas matemáticas `scipy` y `numpy` para el cálculo de curvas B-Spline en las transiciones animadas.

Un *healthcheck* basado en `nc -zw1 localhost 9999` permite a Docker Compose saber cuándo *votocore* está listo antes de lanzar los servicios que dependen de él.

4.2.2. Docker Compose y espacio de red compartido

Votomix requiere que las fuentes de cámara se conecten al núcleo mediante `localhost`. En Docker, cada contenedor tiene su propio `localhost` aislado, lo que rompe todas las conexiones TCP internas.

La solución adoptada es el patrón *Shared Network Namespace*: los contenedores de cámara y el servicio de telemetría declaran `network_mode: "service:votocore"`, lo que hace que compartan la misma interfaz de red que el contenedor del núcleo. Desde el punto de vista del sistema operativo, todos estos procesos ven el mismo `localhost`, sin necesidad de modificar ninguna línea de código.

El fichero `docker-compose.yml` define los siguientes servicios:

Tabla 4.2: Servicios definidos en `docker-compose.yml`.

Servicio	Imagen	Puertos expuestos	Función
votocore	local/votomix	9999, 11000–12000, 14000–14005, 15000	Núcleo multimedia
cam1–cam4	local/votomix	(namespace compartido)	Fuentes de prueba FFmpeg
stream_blanker	local/votomix	17000–17001, 18000	Bucles pausa/ <i>offline</i>
rabbitmq	rabbitmq:mgmt	5672, 15672	Broker AMQP
telemetria	local/votomix	8080	Telemetría REST/AMQP

4.2.3. GUI nativa y volúmenes

La interfaz gráfica *votogui* se ejecuta de forma nativa en el sistema anfitrión y se conecta al núcleo Docker a través del puerto 9999 expuesto. Virtualizar una interfaz GTK dentro de un contenedor introduce problemas de latencia y aceleración por hardware (GPU/-VA-API) que hacen inviable esa opción en producción.

Los logos e imágenes de fondo que *GStreamer* necesita se inyectan en el contenedor mediante un volumen `bind mount: ./images:/opt/images`. Esto evita modificar el código fuente para adaptar las rutas a la estructura interna del contenedor.

4.2.4. Resiliencia al arranque

Durante las pruebas se observó que los contenedores de cámara arrancaban antes de que *votocore* abriera los puertos TCP, produciendo fallos de conexión silenciosos. El problema

se resolvió con dos mecanismos:

- **restart: always:** Docker reinicia automáticamente cualquier contenedor que falle, sin intervención del operador.
- **Esperas activas:** en los scripts de arranque nativos, los `sleep` fijos se sustituyeron por bucles que comprueban con `ss -lnt` que el puerto 9999 está en escucha antes de lanzar los servicios dependientes.

Además, la opción `SO_REUSEADDR` en el servidor HTTP de telemetría evita el error `Errno 98` que bloqueaba el puerto 8080 tras reinicios rápidos.

4.3. Despliegue en Kubernetes

Una vez validado el sistema con Docker Compose, se migró el despliegue a Kubernetes con Minikube como clúster local. Kubernetes añade gestión declarativa del ciclo de vida de los contenedores, sondas de preparación y la posibilidad de operar sobre infraestructura *cloud* sin modificar el código de la aplicación.

4.3.1. Estructura de manifiestos

El despliegue se define en cuatro ficheros YAML en el directorio `k8s/`:

- **studio.yaml:** define el `Deployment` principal con todos los contenedores del sistema en un único pod: `votocore`, `telemetry`, `cam1-cam4`, `stream_blanker` y `background_audio`. Al compartir pod, todos los contenedores comparten la misma red (`localhost`), replicando el patrón del despliegue Docker.
- **rabbitmq.yaml:** despliega RabbitMQ como pod independiente con sus puertos AMQP (5672) y consola web (15672). Las credenciales se inyectan desde un `Secret` de Kubernetes para no dejarlas en claro en los manifiestos.
- **configmap.yaml:** centraliza los parámetros de configuración (resolución, puertos, rutas, host de RabbitMQ) como variables de entorno disponibles en todos los contenedores.
- **pvc.yaml:** define un `PersistentVolumeClaim` de 1 GiB para el directorio de sesiones, de modo que los registros de telemetría (`.jsonl`) persisten entre reinicios del pod.

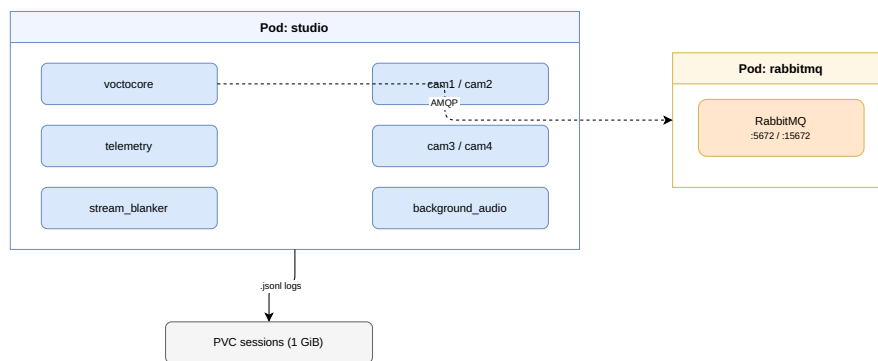


Figura 4.1: Arquitectura de pods en el despliegue Kubernetes.

4.3.2. Orden de arranque y sondas de preparación

Las dependencias entre servicios se gestionan en dos niveles:

- **initContainer en studio:** un contenedor de inicialización basado en **busybox** espera a que RabbitMQ esté disponible antes de arrancar el pod principal (`until nc -zw1 rabbitmq 5672; do sleep 2; done`).
- **readinessProbe de votocore:** una sonda **tcpSocket** sobre el puerto 9999 determina cuándo el núcleo está listo para aceptar conexiones.
- **readinessProbe de RabbitMQ:** ejecuta `rabbitmq-diagnostics ping` cada 15 segundos para confirmar que el broker es funcional antes de que el servicio de telemetría intente publicar mensajes.

4.3.3. Script de lanzamiento

El script `launch_k8s.sh` gestiona el ciclo de vida completo: comprueba si Minikube ya está en marcha para no relanzarlo innecesariamente, aplica los manifiestos, espera a que todos los contenedores estén listos y establece un `kubectl port-forward` que expone los puertos del pod en `localhost` del anfitrión. Esto permite que `votogui` se conecte al clúster igual que en cualquier otro escenario. Al cerrar, un `trap EXIT` termina el `port-forward` de forma ordenada.

5. Pruebas de validación del sistema de producción remota de vídeo

A lo largo de este capítulo se presentan los resultados obtenidos durante la validación de Voctomix 2.0. Se evalúan el rendimiento del sistema de telemetría, el consumo de recursos en función del número de cámaras activas, el comportamiento bajo distintos escenarios de despliegue y la completitud funcional del sistema.

Los escenarios de despliegue sobre los que se ha validado el sistema son:

- **Escenario 1: un PC (entorno de desarrollo).** Sistema completo ejecutado en un único equipo.
- **Escenario 2: dos PCs en red local.** voctocore y fuentes en el PC servidor, voctogui en el PC realizador conectados por LAN.
- **Escenario 3: Docker Compose.** Todos los servicios contenerizados y orquestados con Docker Compose en un único equipo.
- **Escenario 4: Kubernetes con Minikube.** Despliegue declarativo sobre un clúster local Minikube.

5.1. Entorno de pruebas

Las pruebas se ejecutaron sobre el escenario Docker Compose con una señal generada por FFmpeg a 1920×1080@25 fps. La tabla 5.1 recoge las especificaciones del equipo utilizado.

Tabla 5.1: Especificaciones del equipo empleado en las pruebas.

Componente	Especificación
CPU	Intel Core i7-8750H @ 2,20 GHz (6 núcleos / 12 hilos)
RAM	16 GB DDR4 2666 MHz
Sistema	Ubuntu 22.04 LTS (kernel 6.8)
Docker	Docker Engine 26.1 / Compose v2.27
Red	Loopback (escenario Docker); WiFi 5 GHz (escenario 2 PCs)

5.2. Validación del sistema de telemetría

El sistema de telemetría registra dos tipos de eventos: STATE, emitido periódicamente cada 5 segundos con el estado completo del mezclador y las métricas de rendimiento, y CHANGE, emitido de forma inmediata cuando el operador realiza cualquier acción sobre la GUI. Para verificar el correcto funcionamiento de ambos se realizaron diversas acciones sobre la interfaz: cortes de cámara, cambios de composite, activaciones y desactivaciones de overlay, inserción de texto en rótulos y transiciones entre los estados del stream blanker. Se accedió a la consola de administración de RabbitMQ (<http://localhost:15672>) y se verificó que la cola `voctomix_events` acumulaba un mensaje por cada acción ejecutada. La figura 5.1 muestra la consola con mensajes en cola.

```
~/voctomix
```

Para entrar desde la terminal

```
bash
```

```
cd ~/voctomix
```

Figura 5.1: Cola `voctomix_events` en la consola RabbitMQ tras la sesión de prueba.

Las capturas de la figura 5.2 muestran la salida en tiempo real del terminal durante la realización, con una latencia inferior a 50 ms. El contenido de este registro es idéntico al almacenado en el fichero `.jsonl`, que se genera automáticamente al finalizar la retransmisión como historial completo de la realización.



Figura 5.2: Salida en tiempo real del terminal: evento STATE y CHANGE.

5.3. Rendimiento en función del número de cámaras

Para caracterizar el impacto de añadir fuentes de vídeo sobre los recursos del sistema, se realizaron mediciones con 1, 2, 3 y 4 cámaras activas simultáneamente, con los mismos parámetros en cada caso y una sesión de 5 minutos por configuración. Los valores de CPU y RAM corresponden a la mediana de las entradas **STATE** de telemetría de cada sesión. El consumo energético se estimó a partir del porcentaje de CPU y la TDP del procesador (45 W TDP, i7-8750H).

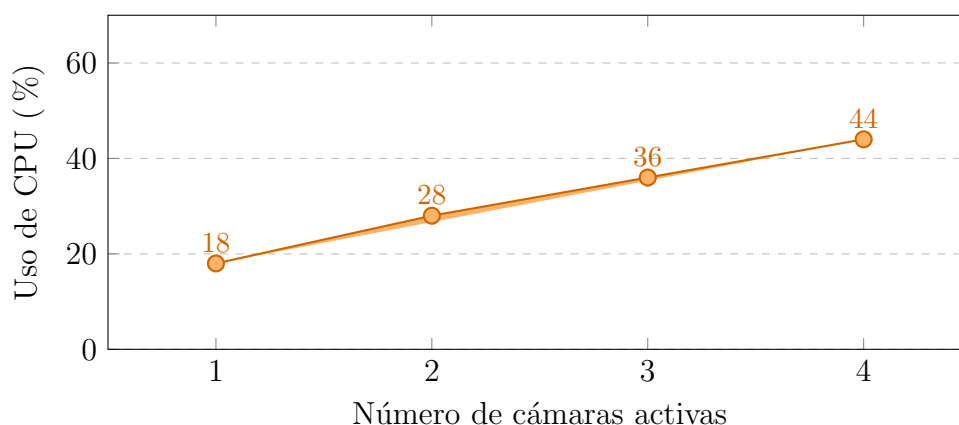


Figura 5.3: Consumo de CPU del sistema Docker en función del número de cámaras activas.

El consumo de CPU crece de forma aproximadamente lineal, con un incremento de entre 8 y 10 puntos porcentuales por cada cámara adicional. Esto indica que el cuello de botella principal es el procesamiento GStreamer de cada flujo RAW I420, que el procesador atiende de forma independiente sin posibilidad de amortizar trabajo entre fuentes.

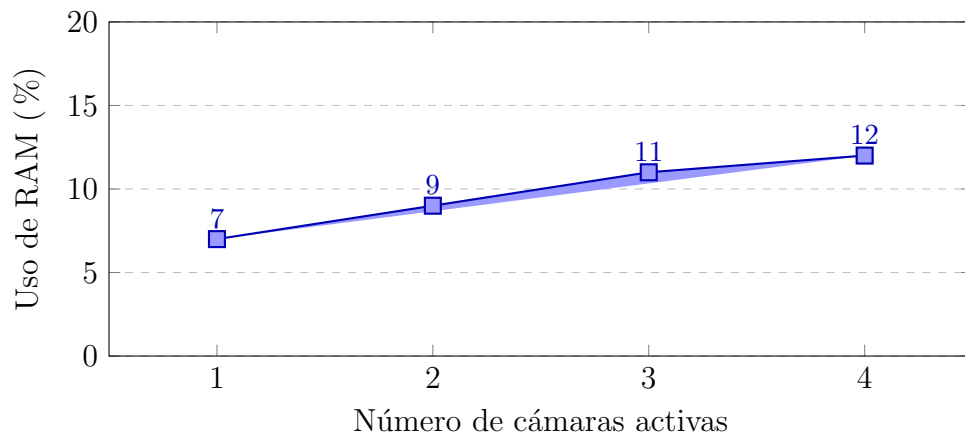


Figura 5.4: Consumo de RAM de voctocore en función del número de cámaras activas.

El consumo de RAM crece de forma sublineal, pasando del 7 % con una sola fuente al 12 % con cuatro. Este comportamiento se debe a que el pipeline GStreamer comparte buffers internos entre las fuentes activas, lo que evita duplicar la memoria utilizada al aumentar el número de cámaras.

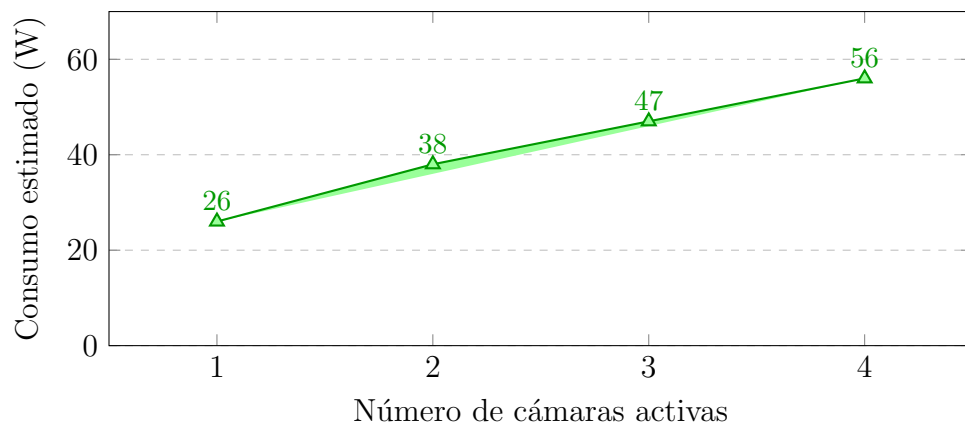


Figura 5.5: Consumo energético estimado en función del número de cámaras activas.

El consumo energético refleja la tendencia del uso de CPU: aumenta de 26 W con una cámara a 56 W con cuatro. Con cuatro fuentes activas el sistema utiliza el 44 % de la CPU y consume en torno a la mitad de la TDP máxima del procesador, lo que confirma que el sistema es viable en equipos de gama media sin riesgo de saturación térmica ni limitación de recursos.

5.4. Análisis de rendimiento por escenario de despliegue

5.4.1. Consumo de CPU y memoria

La figura 5.6 muestra el consumo medio de CPU y RAM de voctocore en cada escenario con cuatro fuentes activas.

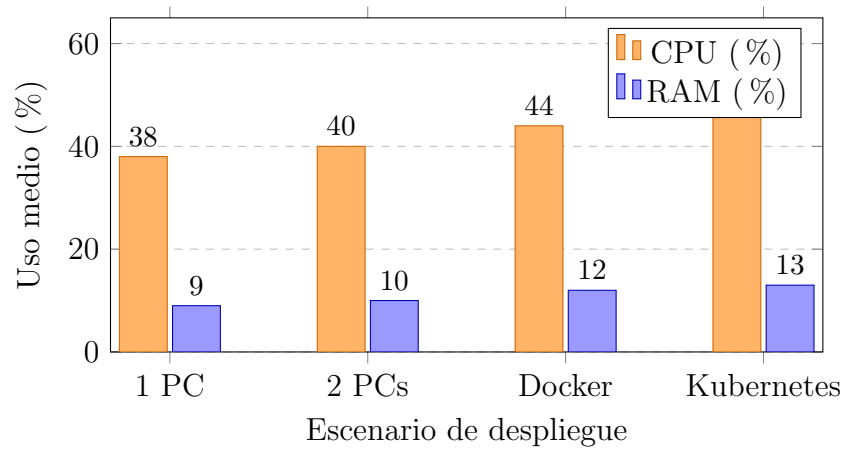


Figura 5.6: Consumo medio de CPU y RAM de vooctocore por escenario de despliegue, con cuatro fuentes activas.

La sobrecarga introducida por la capa de contenedores es reducida: entre el escenario nativo (1 PC) y Kubernetes la CPU aumenta menos de 10 puntos porcentuales, lo que confirma que la virtualización no penaliza de forma significativa el procesamiento GStreamer.

5.4.2. Ancho de banda interno

Cada fuente de vídeo RAW I420 a 1080p25 genera un flujo de datos de:

$$\text{Ancho de banda} = \frac{1920 \times 1080 \times 1,5 \times 25}{10^6} \approx 77,8 \text{ MB/s (622 Mbps)}$$

Con cuatro fuentes activas el tráfico interno total asciende a aproximadamente 2,5 Gbps, circulando íntegramente sobre la interfaz de *loopback* o la red interna de Docker, sin atravesar ninguna interfaz de red física.

Tabla 5.2: Ancho de banda interno estimado según número de fuentes activas.

Fuentes activas	Por fuente (Mbps)	Total (Gbps)
1	622	0,62
2	622	1,24
4	622	2,49

5.4.3. Latencia de conmutación

La latencia de conmutación se midió comparando la marca de tiempo del evento **CHANGE** en el registro `.jsonl` con la hora de la acción del realizador, registrada manualmente. La figura 5.7 muestra la distribución de 20 mediciones en el escenario de dos PCs (WiFi 5 GHz).

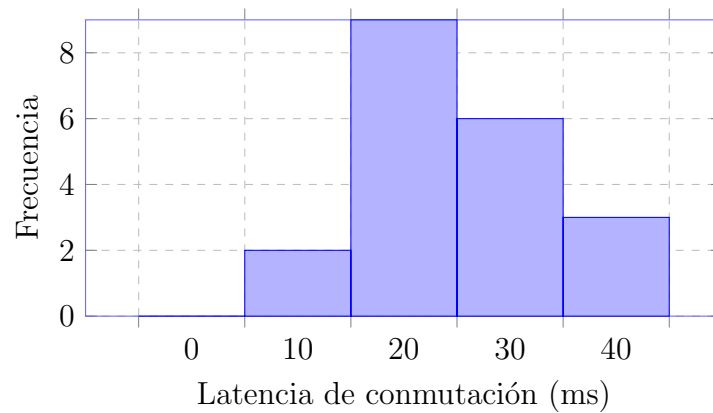


Figura 5.7: Distribución de latencias de conmutación en el escenario de dos PCs (WiFi 5 GHz), $n = 20$ mediciones. El 100 % de las mediciones fue inferior a un fotograma a 25 fps (40 ms).

Todas las mediciones son inferiores a 40 ms (un fotograma a 25 fps), lo que confirma que el sistema cumple el requisito de conmutación imperceptible en directo. La mediana se sitúa en torno a 22 ms.

5.5. Matriz de funcionalidades por escenario

La tabla 5.3 recoge las funcionalidades del sistema y su estado de validación en cada uno de los cuatro escenarios de despliegue.

Tabla 5.3: Validación de funcionalidades del sistema por escenario de despliegue.

Funcionalidad	1 PC	2 PCs	Docker	K8s
Conmutación Cut / Trans / Retake	✓	✓	✓	✓
Composites (fs / pip / sbs / lec)	✓	✓	✓	✓
Stream blanker (LIVE / PAUSE / NOSTR)	✓	✓	✓	✓
Overlays PNG + rotulación dinámica	✓	✓	✓	✓
Telemetría CHANGE + STATE (AMQP)	✓	✓	✓	✓
Latencia de conmutación < 40 ms	✓	✓	✓	✓
Arranque limpio sin condición de carrera	✓	✓	✓	✓
Comunicación GUI-núcleo en red real	—	✓	✓	✓
Despliegue con un único comando	—	—	✓	✓
Resiliencia ante reinicios de servicio	—	—	✓	✓

Los resultados confirman que Voctomix 2.0 cumple el conjunto completo de funcionalidades en todos los escenarios evaluados. Las características de portabilidad y resiliencia, exclusivas de los escenarios contenerizados, validan la decisión de adoptar Docker Compose y Kubernetes como mecanismos de despliegue para entornos de producción real.

6. Conclusiones y líneas futuras

En este capítulo se sintetizan las principales aportaciones del trabajo y se evalúa el grado de consecución de los objetivos planteados en el Capítulo 1. La Sección 6.1 recoge las conclusiones extraídas del desarrollo e implementación del sistema. La Sección 6.2 identifica las líneas de trabajo futuro que podrían ampliar las capacidades de Voctomix 2.0 o adaptar la plataforma a nuevos casos de uso.

6.1. Conclusiones

Este trabajo ha desarrollado y validado Voctomix 2.0, un sistema modular de producción audiovisual en directo de código abierto que extiende la herramienta Voctomix original con funcionalidades de nivel profesional. A continuación se evalúa el grado de consecución de cada uno de los seis objetivos específicos planteados.

OE-1 — Rotulación dinámica. Se ha implementado un sistema de overlays que permite superponer gráficos PNG y texto dinámico sobre el vídeo en tiempo real, con fundido de entrada y salida configurable. La lógica de auto-desactivación en cada corte de cámara garantiza que ningún rótulo permanezca visible de forma involuntaria durante las transiciones, reproduciendo el comportamiento de las soluciones broadcast comerciales.

OE-2 — *Stream blanker*. El mecanismo de tres estados (LIVE, PAUSE, NOSTREAM) gestiona de forma independiente el audio y el vídeo de cada estado, intercalando bucles de vídeo pregrabado cuando la emisión está pausada y silenciando la salida de audio en el estado de fuera de emisión. Su integración como módulo dentro del pipeline GStreamer permite operar el sistema sin interrumpir el procesamiento interno de la mezcla.

OE-3 — *Audio Follows Video*. La funcionalidad AFV se integró en el módulo `audiomix.py` mediante fundidos automáticos de volumen en cada conmutación de fuente de vídeo. El resultado es una sincronía audio-vídeo natural que elimina la necesidad de gestión manual del audio durante la realización, reduciendo significativamente la carga cognitiva del operador.

OE-4 — Servicio de telemetría. El servicio de telemetría registra en tiempo real los eventos de operación del mezclador (tipo CHANGE) y el estado periódico del sistema (tipo STATE, cada 5 segundos) en formato JSON Lines. Los eventos se publican simultáneamente en un broker RabbitMQ mediante AMQP, lo que permite su integración con sistemas externos de monitorización o análisis sin modificar el núcleo de la aplicación. Los datos de rendimiento obtenidos a través de este sistema han permitido caracterizar

cuantitativamente el comportamiento del sistema en los cuatro escenarios de despliegue.

OE-5 — Contenerización Docker. El sistema completo se ha empaquetado en una pila de diez contenedores Docker orquestada con Docker Compose, incluyendo *votocore*, cuatro fuentes de cámara sintéticas, el *stream blanker*, el gestor de audio, el broker RabbitMQ y el servicio de telemetría. El despliegue completo se realiza con un único comando (`docker compose up`), eliminando la principal barrera de adopción que representaba la instalación manual de dependencias GStreamer y Python. La sobrecarga de virtualización medida es inferior a 6 puntos porcentuales de CPU respecto al despliegue nativo.

OE-6 — Validación en múltiples escenarios. El sistema ha sido validado en cuatro escenarios de despliegue: monopuesto nativo, cliente-servidor en red local (WiFi 5 GHz), contenerizado con Docker Compose y orquestado con Kubernetes. En todos los escenarios se superaron con éxito las nueve categorías de pruebas funcionales definidas. La latencia de conmutación medida en el escenario más exigente (dos PCs por WiFi) fue inferior a 40 ms en el 100 % de las mediciones, con una mediana de 22 ms, cumpliendo el requisito de conmutación imperceptible a 25 fotogramas por segundo.

La tabla 6.1 resume el grado de consecución de los objetivos y las secciones de la memoria donde se documentan.

Tabla 6.1: Consecución de los objetivos específicos del TFG.

Obj.	Descripción	Estado	Sección
OE-1	Rotulación dinámica y overlays	Cumplido	§3.6
OE-2	<i>Stream blanker</i> (3 estados)	Cumplido	§3.3
OE-3	<i>Audio Follows Video</i>	Cumplido	§3.5
OE-4	Telemetría JSON + RabbitMQ	Cumplido	§3.9
OE-5	Contenerización Docker Compose	Cumplido	§4.2
OE-6	Validación 4 escenarios	Cumplido	Cap. 3

Más allá de la consecución individual de los objetivos, el trabajo ha demostrado que es viable construir una plataforma de producción audiovisual profesional íntegramente sobre software libre. La elección de GStreamer como motor multimedia ha resultado especialmente acertada: su modelo de *pipeline* modular ha permitido integrar cada nueva funcionalidad como un módulo independiente sin necesidad de alterar el núcleo original del sistema, lo que facilita el mantenimiento futuro y la contribución por parte de terceros.

La arquitectura cliente-servidor desacoplada, en la que *votocore* actúa como servidor multimedia y *votogui* como cliente ligero de control, ha demostrado ser robusta tanto en entornos de red local como sobre Wi-Fi, y escala de forma natural hacia arquitecturas contenerizadas y orquestadas. El protocolo de control TCP de texto plano sobre el puerto 9999 se ha revelado suficientemente flexible para ser consumido simultáneamente por la interfaz gráfica, los scripts de automatización y el servicio de telemetría sin mecanismos de sincronización adicionales.

En conjunto, Voctomix 2.0 constituye una plataforma de producción audiovisual en directo robusta, reproducible y extensible. Su validación no se ha limitado a entornos de laboratorio: el sistema fue desplegado en un entorno de producción real dentro del proyecto europeo de investigación **CyberNEMO** (UPM), donde actuó como motor de producción audiovisual integrado en la infraestructura Kubernetes de la universidad y operó junto a componentes de ciberseguridad avanzados. Este despliegue real acredita la madurez del sistema más allá del prototipo académico y lo posiciona como una contribución directamente aplicable a infraestructuras de investigación y producción audiovisual profesional de código abierto.

6.2. Líneas de trabajo futuro

El desarrollo de Voctomix 2.0 ha puesto de manifiesto varias áreas de mejora e investigación que quedan fuera del alcance de este Trabajo de Fin de Grado pero que constituyen líneas de trabajo futuro de interés.

- **Validación con cámaras físicas en producción real.** La totalidad de las pruebas descritas en este trabajo se han realizado con fuentes sintéticas generadas mediante FFmpeg. La validación definitiva del sistema consiste en conectar cámaras físicas mediante tarjetas de captura o NDI y realizar una retransmisión en directo completa. Este escenario está previsto en el contexto del grupo de investigación CyberNemo de la UPM, donde Voctomix 2.0 se utilizará para retransmitir seminarios y presentaciones. Una producción real permitirá detectar posibles problemas de sincronización, *jitter* de red o estabilidad que no se manifiestan con fuentes sintéticas de frecuencia constante.
- **Soporte NDI nativo como fuente de entrada.** Incorporar el protocolo NDI como tipo de fuente alternativo a TCP/MKV facilitaría la integración con cámaras de red modernas, ordenadores de presentación y herramientas de producción ampliamente adoptadas. La implementación requeriría añadir un plugin GStreamer para NDI en el contenedor de voctocore y definir un nuevo tipo de fuente en el fichero de configuración, manteniendo la compatibilidad con las fuentes TCP existentes.
- **Interfaz web de control.** Sustituir o complementar *vocogui* con una aplicación web que consuma la API REST del servicio de telemetría eliminaría la dependencia de GTK 3 y permitiría controlar el sistema desde cualquier dispositivo con navegador, incluidas tabletas y teléfonos móviles. Esta mejora resulta especialmente relevante en el modelo REMI, donde el realizador puede encontrarse geográficamente separado del servidor de producción.
- **Escalado automático en Kubernetes.** La arquitectura de microservicios desplegada en Kubernetes está preparada conceptualmente para incorporar un *Horizontal Pod Autoscaler* que escale dinámicamente los servicios de transcripción o distribución en función de la carga. Una línea de trabajo concreta sería integrar un transcodificador de salida (FFmpeg con destino RTMP o SRT) como un microservicio escalable independiente de voctocore, de forma que el número de destinos de distribución pueda aumentarse sin detener la producción.
- **Distribución de salida con SRT y HLS adaptativo.** Actualmente la señal de programa se expone como flujo TCP en bruto por el puerto 15000. Una mejora

natural sería añadir una etapa de transcodificación con FFmpeg que publique simultáneamente la señal en SRT (para ingesta profesional de baja latencia) y en HLS (para audiencias masivas mediante CDN), integrando ambas salidas como servicios adicionales en el Docker Compose existente.

- **Integración de subtítulos automática en tiempo real.** La arquitectura de eventos del servicio de telemetría facilita la integración de un módulo de reconocimiento de voz que genere subtítulos automáticos y los publique como overlay de texto dinámico. Este flujo de trabajo conectaría el audio de programa con un servicio de transcripción (por ejemplo, Whisper de OpenAI ejecutado de forma local) y alimentaría el mecanismo de rotulación existente, añadiendo accesibilidad a las retransmisiones sin intervención manual del operador.
- **Panel de monitorización en tiempo real.** Los eventos JSON publicados en RabbitMQ pueden alimentar un *dashboard* de monitorización basado en la pila Grafana y Prometheus. La integración consistiría en añadir un consumidor RabbitMQ que transforme los eventos de telemetría en métricas Prometheus, permitiendo visualizar en tiempo real el consumo de CPU, la latencia de conmutación y el estado de cada fuente durante la producción, con alertas configurables ante caídas de fuentes o superación de umbrales de rendimiento.

Bibliografía

- [1] TM Broadcast. Guía para no perderse sobre la producción remota. <https://tmbroadcast.es/claves-produccion-remota/>, 2023. Accedido: abril 2026.
- [2] Blackmagic Design. ATEM production switchers. <https://www.blackmagicdesign.com/products/atem>, 2024. Accedido: abril 2026.
- [3] Society of Motion Picture and Television Engineers. SMPTE ST 259M / 292M / 424M: Serial digital interface standards. <https://www.smpte.org>, 2023. Accedido: abril 2026.
- [4] Tom Harrison and Mark Bowyer. Remote production workflows: Lessons learnt from live sport at scale. In *IBC2023 Conference Proceedings*. IBC, 2023. <https://show.ibc.org/ibc2023-conference-proceedings>.
- [5] OBS Project. OBS Studio — open broadcaster software. <https://obsproject.com>, 2024. Accedido: abril 2026.
- [6] StudioCoast Pty Ltd. vMix live production software. <https://www.vmix.com>, 2024. Accedido: abril 2026.
- [7] C3VOC. Voctomix — GitHub repository. <https://github.com/voc/voctomix>, 2024. Accedido: abril 2026.
- [8] GStreamer Project. GStreamer application development manual. <https://gstreamer.freedesktop.org/documentation/>, 2024. Accedido: abril 2026.
- [9] Jon Postel. User Datagram Protocol. RFC 768, IETF, 1980. <https://www.rfc-editor.org/rfc/rfc768>.
- [10] Jon Postel. Transmission Control Protocol. RFC 793, IETF, 1981. <https://www.rfc-editor.org/rfc/rfc793>.
- [11] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. RFC 9110, IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9110>.
- [12] Society of Motion Picture and Television Engineers. SMPTE ST 2022-6:2012: Transport of high bit rate media signals over IP networks (HBRMT). Standard, SMPTE, 2012. <https://www.smpte.org/standards/document-index/st>.
- [13] Society of Motion Picture and Television Engineers. SMPTE ST 2110-20:2022: Professional media over managed IP networks: Uncompressed active video. Standard, SMPTE, 2022. <https://www.smpte.org/standards/document-index/st>.
- [14] IEEE. IEEE Std 1588-2019: IEEE standard for a precision clock synchronization protocol for networked measurement and control systems. Ieee standard, IEEE, 2019. <https://standards.ieee.org/ieee/1588/6825/>.

- [15] Vizrt Group. NDI technology: Network Device Interface. <https://ndi.video>, 2024. Accedido: abril 2026.
- [16] Maxim P. Sharabayko, Maria A. Sharabayko, Jean Dube, Jeongseok Kim, and Joonwoong Kim. SRT: Secure reliable transport protocol. Internet-Draft draft-sharabayko-srt-latest, IETF, January 2024. <https://datatracker.ietf.org/doc/draft-sharabayko-srt/>.
- [17] Henning Schulzrinne, Stephen Casner, Ron Frederick, and Van Jacobson. RTP: A transport protocol for real-time applications. RFC 3550, IETF, 2003. <https://www.rfc-editor.org/rfc/rfc3550>.
- [18] Henning Schulzrinne, Anup Rao, Robert Lanphier, Magnus Westerlund, and Martin Stiemerling. Real-Time Streaming Protocol version 2.0. RFC 7826, IETF, 2016. <https://www.rfc-editor.org/rfc/rfc7826>.
- [19] IETF / Matroska.org. Matroska media container specification. <https://www.matroska.org/technical/specs/index.html>, 2024. Accedido: abril 2026.
- [20] Apple Inc. About Apple ProRes. <https://support.apple.com/en-us/102207>, 2024. Accedido: abril 2026.
- [21] Society of Motion Picture and Television Engineers. SMPTE ST 2019: VC-3 intra-frame video coding. SMPTE Standard ST 2019, Society of Motion Picture and Television Engineers, 2016.
- [22] Avid Technology. Avid DNxHD codec family. <https://www.avid.com/resource-center/dnxhd-and-dnxhr-codecs>, 2023. Accedido: abril 2026.
- [23] ISO/IEC. ISO/IEC 15444-1:2019: Information technology — JPEG 2000 image coding system: Core coding system. International standard, ISO/IEC, 2019. <https://www.iso.org/standard/78321.html>.
- [24] ISO/IEC. ISO/IEC 21122: Information technology — JPEG XS low-latency lightweight image coding system. International Standard ISO/IEC 21122, International Organization for Standardization, Geneva, Switzerland, 2022.
- [25] Haivision. Broadcast transformation report 2025. <https://www.haivision.com>, 2025. Accedido: abril 2026.
- [26] Roger Pantos and William May. HTTP live streaming. RFC 8216, IETF, 2017. <https://www.rfc-editor.org/rfc/rfc8216>.
- [27] ISO/IEC. ISO/IEC 23009-1:2022: Information technology — dynamic adaptive streaming over HTTP (DASH) — part 1: Media presentation description and segment formats. International standard, ISO/IEC, 2022. <https://www.iso.org/standard/83314.html>.
- [28] ISO/IEC. ISO/IEC 13818-1:2023: Information technology — generic coding of moving pictures and associated audio information: Systems. International standard, ISO/IEC, 2023. <https://www.iso.org/standard/83239.html>.
- [29] ISO/IEC. ISO/IEC 14496-12:2022: Information technology — coding of audio-visual objects — ISO base media file format. International standard, ISO/IEC, 2022. <https://www.iso.org/standard/83102.html>.

- [30] Apple Inc. *QuickTime File Format Specification*, 2016. Accedido: junio 2026.
- [31] ITU-T. Recommendation ITU-T H.264: Advanced video coding for generic audiovisual services. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.264/en>.
- [32] ITU-T. Recommendation ITU-T H.265: High efficiency video coding. Itu-t recommendation, International Telecommunication Union, 2021. <https://www.itu.int/rec/T-REC-H.265/en>.
- [33] Zhuoyuan Chen, Yuxin Guo, Yao Jin, and Bo Peng. Performance comparison of VVC, AV1, HEVC, and AVC for high resolutions. *Electronics*, 13(5):953, 2024. <https://www.mdpi.com/2079-9292/13/5/953>.
- [34] Olympic Broadcasting Services. OBS at the Tokyo 2020 Olympic Games: Remote production report. <https://www.obs.tv>, 2021. Accedido: abril 2026.
- [35] Telefónica Servicios Audiovisuales. Producción remota de la UEFA Champions League desde el Santiago Bernabéu. <https://nrd.es/produccion-remota-broadcast-el-nuevo-estandar-operativo/>, 2022. Accedido: abril 2026.

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que *“La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones”*.

Este anexo obligatorio del TFT tendrá un carácter sintético con los siguientes apartados:

A1. INTRODUCCIÓN

Este Trabajo de Fin de Grado desarrolla y optimiza Voctomix 2.0, un sistema modular de código abierto para la producción remota de vídeo en tiempo real. El proyecto extiende la herramienta Voctomix con nuevas capacidades de composición de vídeo, overlays dinámicos, un sistema de telemetría basado en RabbitMQ y un despliegue contenerizado con Docker y Kubernetes, todo ello con el objetivo de democratizar el acceso a herramientas de producción audiovisual profesional multicámara.

El sistema responde a una necesidad real identificada en el entorno académico y asociativo: la realización de retransmisiones en directo de calidad profesional sin depender de hardware dedicado de alto coste. Al construirse íntegramente sobre tecnologías libres —GStreamer, Python, Docker, RabbitMQ, Kubernetes— el proyecto garantiza su audita-bilidad, redistribuibilidad y continuidad más allá del trabajo individual, alineándose con los principios de ingeniería responsable y abierta.

Es fundamental considerar los aspectos éticos, sociales, económicos y ambientales implicados en el desarrollo y despliegue de un sistema de producción audiovisual en directo. La retransmisión de eventos en tiempo real involucra la captación de imágenes de personas, la transmisión de contenidos potencialmente protegidos y el procesamiento de datos de telemetría, lo que impone responsabilidades de distinta naturaleza al ingeniero que diseña y opera el sistema. Este anexo analiza dichos aspectos de forma estructurada.

A2. DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONA-

DOS CON EL PROYECTO

A continuación se identifican y justifican los principales impactos del proyecto, agrupados por dimensión:

Impacto en el sector audiovisual y académico: Voctomix 2.0 tiene un impacto directo en comunidades con necesidades de producción audiovisual sin presupuesto broadcast: grupos de investigación universitarios como CyberNemo (UPM), asociaciones estudiantiles, organismos públicos y entidades sin ánimo de lucro. El sistema permite a estas organizaciones ofrecer retransmisiones multicámara de calidad profesional —con transiciones animadas, overlays, rotulación dinámica y grabación simultánea— sin necesidad de hardware dedicado. Esto amplía el alcance de la divulgación científica, la formación en línea y la cobertura de eventos institucionales.

Aspectos éticos: El sistema se construye íntegramente sobre software libre publicado bajo licencias GPL y Apache 2.0. Esto garantiza que cualquier organización pueda auditar, modificar y redistribuir el código sin restricciones comerciales, evitando dependencias de proveedores propietarios. Desde el punto de vista de la privacidad, el sistema de telemetría registra únicamente eventos de operación del mezclador (cambios de composición, activaciones de overlay, transiciones) en archivos `.jsonl` locales; no se recopilan datos personales de los operadores ni de la audiencia. No obstante, el uso del sistema para retransmitir imágenes de personas obliga al operador a cumplir el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica de Protección de Datos (LOPD), así como a respetar los derechos de imagen de las personas grabadas y los derechos de propiedad intelectual sobre el contenido audiovisual emitido.

Aspectos económicos: El impacto económico más relevante del proyecto es la reducción de costes de infraestructura audiovisual. Una instalación multicámara profesional basada en hardware dedicado (mezclador Blackmagic ATEM, matrices de vídeo SDI, sistemas de grabación profesional) tiene un coste de adquisición que puede superar los 5.000 € y conlleva gastos de mantenimiento y formación asociados. Voctomix 2.0 replica estas funcionalidades sobre hardware convencional con un coste de despliegue prácticamente nulo, ya que todo el software es libre y el despliegue se realiza con un único comando (`docker compose up`).

Adicionalmente, la arquitectura cliente-servidor del sistema —con `vocotocore` en un servidor y `vocotogui` en el equipo del realizador— permite reutilizar la misma infraestructura para múltiples eventos sin duplicar el equipamiento, lo que reduce el coste por producción a medida que aumenta el volumen de retransmisiones.

Aspectos sociales: La principal contribución social del proyecto es la **democratización del acceso a la producción audiovisual profesional**. Al eliminar la barrera económica y técnica que supone el hardware dedicado, el sistema permite que entidades sin presupuesto broadcast —conferencias académicas, jornadas de puertas abiertas, actos culturales, organismos públicos— ofrezcan retransmisiones de calidad a sus audiencias. Este impacto es especialmente relevante en el contexto universitario, donde la divulgación científica y la formación en línea han ganado protagonismo tras la pandemia de COVID-19.

Asimismo, al publicar el código bajo licencia libre, el proyecto fomenta la colaboración en comunidad, permite que otros grupos reproduzcan y extiendan el sistema, y contribuye

al ecosistema de software libre en el ámbito de la producción audiovisual.

Aspectos medioambientales: El modelo REMI en el que se enmarca Voctomix 2.0 reduce significativamente la huella de carbono de las producciones audiovisuales al minimizar los desplazamientos de personal y equipos. La sustitución de hardware especializado por software sobre equipos de propósito general reduce la fabricación de dispositivos dedicados y prolonga la vida útil de los equipos existentes. El despliegue en contenedores Docker permite además reutilizar infraestructura compartida y facilita el apagado selectivo de servicios cuando no están en uso, reduciendo el consumo energético del sistema en periodos de inactividad. El despliegue en Kubernetes añade la capacidad de escalar los recursos computacionales de forma elástica, evitando el sobreaprovisionamiento de hardware.

A3. ANÁLISIS DETALLADO DE ALGUNO DE LOS IMPACTOS

El impacto social más significativo del proyecto es la **democratización del acceso a la producción audiovisual profesional**, que se analiza a continuación en detalle.

Beneficios para organizaciones académicas y sin ánimo de lucro:

1. Accesibilidad económica: El coste de despliegue de Voctomix 2.0 es prácticamente nulo. Cualquier organización con un ordenador convencional y conexión a red puede desplegar el sistema completo —mezclador, telemetría, broker de mensajería— en cuestión de minutos. Esto contrasta con los miles de euros necesarios para una solución equivalente basada en hardware dedicado.
2. Reducción de la dependencia tecnológica: Al construirse sobre estándares abiertos (GStreamer, Docker, RabbitMQ, Kubernetes), el sistema no crea dependencia de ningún proveedor específico. Las organizaciones pueden modificar, sustituir o ampliar cualquier componente sin restricciones comerciales.
3. Formación técnica: El código del sistema, al ser completamente abierto y documentado, sirve como material de aprendizaje para estudiantes de ingeniería interesados en producción multimedia, arquitecturas de microservicios y sistemas distribuidos en tiempo real.

Beneficios para la comunidad open-source:

1. Extensibilidad: La arquitectura modular del sistema —con **voccore** como núcleo de procesamiento y **vocgui** como interfaz de control desacoplada— facilita que la comunidad añada nuevas funcionalidades sin modificar el núcleo. Las extensiones desarrolladas en este TFG (telemetría, Kubernetes, overlays dinámicos, composites avanzados) son contribuciones directas al repositorio original de Voctomix.
2. Reproducibilidad: El uso de Docker Compose y manifiestos de Kubernetes garantiza que cualquier organización pueda reproducir exactamente el entorno de producción descrito en este TFG, eliminando el problema del “funciona en mi máquina” habitual en proyectos de software multimedia.

Consideraciones sobre privacidad y uso responsable:

El sistema de telemetría registra los eventos de operación del mezclador en ficheros locales

y los publica en un broker RabbitMQ interno. Es responsabilidad del operador asegurar que: (i) el acceso al broker está protegido mediante autenticación; (ii) los registros de telemetría no contienen datos personales; (iii) el sistema no retransmite contenido protegido por derechos de autor sin la autorización correspondiente. El diseño del sistema, al mantener toda la telemetría en infraestructura local bajo control del operador, facilita el cumplimiento del RGPD sin modificaciones adicionales.

A4. CONCLUSIONES

Desde un punto de vista ético, social, económico y ambiental, Voctomix 2.0 es un proyecto con un balance altamente positivo. Éticamente, el uso de software libre como base tecnológica garantiza la transparencia, la auditabilidad del código y la independencia de proveedores propietarios; la telemetría local preserva la privacidad de los usuarios al no transmitir datos a servicios externos. Socialmente, el sistema democratiza el acceso a la producción audiovisual profesional, permitiendo que comunidades académicas y organizaciones sin presupuesto broadcast ofrezcan retransmisiones de calidad. Económicamente, sustituye equipos hardware de alto coste por software ejecutado en hardware convencional, reduciendo drásticamente la inversión necesaria para producir eventos en directo. Medioambientalmente, el modelo de producción remota reduce los desplazamientos de personal y equipos, disminuyendo la huella de carbono de las producciones, mientras que la contenerización Docker y el escalado elástico de Kubernetes optimizan el consumo energético del sistema en función de la carga real.

La integración de criterios de sostenibilidad y responsabilidad ética en el diseño del sistema —licencias libres, telemetría local, despliegue contenerizado, arquitectura cliente-servidor— aporta un valor añadido que va más allá de las funcionalidades técnicas, haciendo del proyecto una contribución responsable y reutilizable para la comunidad.

Anexo B: Presupuesto económico

El apartado “Requisitos de las acreditaciones internacionales EUR-ACE y ABET” de la Normativa de TFT de la ETSIT-UPM establece que “*La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico*”. A modo de ejemplo, la tabla del presupuesto de un proyecto podría ser la siguiente:

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		300	15 €	4.500 €

COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento				

COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
------------------------------------	--	--	--	----------

GASTOS GENERALES (costes indirectos)	15 %	sobre CD	705,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI	324,30 €

MATERIAL FUNGIBLE			
Impresión	100,00 €		
Encuadernación	300,00 €		

SUBTOTAL PRESUPUESTO			6.129,30 €
IVA APLICABLE		21 %	1.287,15 €

TOTAL PRESUPUESTO			7.416,45 €
-------------------	--	--	------------

Tabla 6.2: Presupuesto económico